

RESEARCH

Open Access



From chaos to clarity: unraveling credit card fraud with BGVOA-LS

Aya H. Salem^{1*}, Safaa M. Azzam¹, O. E. Emam¹ and Amr A. Abohany²

*Correspondence:
aya.salem@fci.helwan.edu.eg

¹ Faculty of Computer
and Artificial Intelligence, Helwan
University, Cairo, Egypt

² Faculty of Computer
and Information, Damanhour
University, Damanhour, Egypt

Abstract

Credit card fraud (CCF) continues to cause significant financial losses worldwide, costs institutions and consumers billions of dollars each year and erodes confidence in digital payments. Fraudulent activities are rare, highly adaptive, and increasingly complex, making them difficult to detect with traditional rule-based or static models. This study presents the binary Griffon vulture optimization algorithm with local search (BGVOA-LS) framework, which integrates optimized feature selection (FS) with machine learning (ML) to improve accuracy and efficiency in fraud identification by selecting the most relevant transaction features and removing noise and redundancy so that classifiers focus on patterns most indicative of fraudulent behavior. To address the severe imbalance between legitimate and fraudulent records, the method applies random under sampling (RUS), which helps reduce the number of false positives and missed detections. The selected features are evaluated via three classifiers across the European, Australian, and PaySim datasets: decision tree (DT), k-nearest neighbors (KNN), and extreme gradient boosting tree (Xgb-tree). BGVOA-LS achieves up to 99.8% accuracy, reduces the feature count by 67%, and outperforms other metaheuristic techniques (MHTs), enabling faster screening, lower computational costs, and more dependable fraud prevention.

Keywords: Credit card, Fraud detection, Metaheuristic optimization algorithms, Feature selection, Machine learning, Cyber attacks

Introduction

Online payments Have rapidly transformed global commerce, offering unprecedented convenience and efficiency. The growth of e-commerce and digital transactions Has been exponential in recent years; however, this expansion has also intensified challenges related to fraud prevention. In 2024 alone, consumers reported losses exceeding USD 12.5 billion due to various scams, representing a 25% increase from the previous year [1]. The digital payments market is set to hit \$20.37 trillion in 2025, growing at an average rate of 15.9% annually to reach \$36.75 trillion by 2029 [2].

Credit card fraud detection (CCFD) is particularly challenging because of two persistent issues: (i) the constantly evolving tactics of fraudsters, which render static, rule-based systems ineffective [3], and (ii) the severe class imbalance in benchmark datasets, where fraudulent transactions constitute only a small fraction of total transactions. For

example, fraud rates are as low as 0.172% in the European dataset [4] and 0.1% for the PaySim dataset [5] and as High as 44.49% for the Australian dataset [6]. While these percentages appear small in large-scale systems, e.g., 0.172% of one billion daily transactions, this translates to 1.72 million fraudulent cases, each with substantial financial and operational impact. In such imbalanced scenarios, conventional ML models tend to be biased toward the class majority, achieving high apparent accuracy while failing to detect rare but high-impact events [7]. Addressing this imbalance is therefore critical. MHT offers robust solutions for complex optimization problems by efficiently exploring an ample search space [8–10]. MHTs can significantly improve FS, model training and the overall fraud detection process, resulting in more accurate and adaptable techniques [11]. When integrated with ML, these algorithms can produce more accurate, robust, and adaptable detection systems [12, 13].

These challenges are countered by presenting a novel BGVOA-LS-based framework integrated with ML for CCFD, which is designed to improve detection accuracy, adaptability and resilience in dynamic financial environments. The algorithm incorporates RUS to mitigate class imbalance and embeds a local search (LS) mechanism into the original GVOA to maintain population diversity and enhance global exploration. This design enables more effective FS, improving classification accuracy and reducing false positives in highly imbalanced environments.

The main key contributions are as follows:

- We propose a new FS method, BGVOA-LS, tailored for credit card fraud detection.
- To address data imbalance, we integrate BGVOA-LS with RUS to reduce bias toward majority classes in imbalanced datasets.
- Through extensive evaluation, the framework is tested on real-world credit card datasets via multiple classifiers and diverse metrics (AUC, F1 score, MCC, etc.) to ensure robust validation.
- The results demonstrate high predictive accuracy with a substantial reduction in feature size, highlighting both effectiveness and computational efficiency.
- Practically, by combining optimization and resampling techniques the BGVOA-LS algorithm provides a scalable and adaptable approach that can strengthen fraud detection systems in real-world financial environments.

The remainder of the paper is structured as follows: Sect. "Related work" reviews recent studies related to CCFD; Sect. "The proposed methodology" details the proposed BGVOA-LS framework and FS process; and Sect. "Results and analysis" presents and discusses the experimental results. Sect. "Conclusion and future work" explains the conclusions in addition to future research directions.

Motivation

Credit-related predictive tasks such as fraud detection, credit risk assessment and credit outlook prediction operate in high-dimensional, imbalanced, and evolving data environments. These conditions, coupled with real-time performance demands, require solutions that balance accuracy, efficiency, and adaptability [14, 15].

State-of-the-art ML models, including linear discriminant analysis (LDA), logistic regression (LR), DT, support vector machine (SVM), gradient boosting machine (GBM), extreme gradient boosting (XGBoost), deep neural network, and ensemble methods have achieved strong results in financial prediction tasks [16]. However, their performance can be hindered by redundant or noisy features, sensitivity to class imbalance, limited interpretability, and reduced adaptability to shifting data patterns. FS mitigates these challenges by retaining only the most relevant features, thereby improving generalizability, interpretability and computational efficiency.

MHTs have been widely applied for FSs because of their ability to explore large search spaces effectively [17]. However, many methods suffer from premature convergence, high computational cost, and limited refinement in later search stages. The GVOA offers a strong exploration and exploitation balance but lacks mechanisms for targeted local refinement and imbalance handling, both of which are critical in high-stakes financial prediction [18].

To address these gaps, we propose BGVOA-LS, a binary-adapted GVOA enhanced with the following:

- LS for precise subset refinement avoids suboptimal convergence.
- RUS to mitigate class imbalance and improve minority class detection.

To ensure fair and comprehensive evaluation, we selected DT, KNN, and XGB-Tree as representative classifiers. DT offers interpretability value in regulated financial settings; KNN, as an instance-based learner, is highly sensitive to feature relevance, making it a strong FS benchmark; and XGB-Tree represents state-of-the-art ensemble methods with proven success in credit risk and fraud detection. Testing across these diverse learning paradigms confirms the classifier-agnostic effectiveness of BGVOA-LS.

Related work

A wide range of advanced methods have been developed to detect and prevent CCF. This section reviews the latest and most impactful research in CCFD, focusing on techniques such as ML, neural networks (NNs), and deep learning (DL). These approaches also integrate MHTs to increase detection accuracy and system resilience. Effective fraud detection (FD) has become more critical as the volume and complexity of online transactions increase. Researchers have combined ML models with MHTs such as swarm intelligence and evolutionary algorithms to address these evolving threats. These combinations help optimize feature selection FS, fine-tune model parameters, and manage imbalanced datasets [19–21]. Techniques such as DL have demonstrated a strong ability to detect and manage complex patterns in transaction data automatically. Recent advancements have introduced powerful hybrid models such as CNN-LSTM, autoencoders and GANs, significantly improving fraud detection accuracy. New techniques such as federated learning, advanced resampling methods, and reinforcement learning are becoming more active in exploring the scalability, privacy, and adaptability of FD systems; in addition to these enormous developments,

ongoing challenges require continuous research and enhancement. High computational requirements and difficulties in real-world implementation remain key obstacles, encouraging continuing research and innovation in this area [22].

Recent studies have illustrated the trend of integrating MHTs with ML and DL frameworks to enhance detection capabilities and handle imbalanced datasets. For example, Yi et al. (2023) proposed a novel FD model for capital markets via a nonlinear machine learning framework optimized with the egret swarm optimization algorithm (ESOA). The model targets the classification of Highly imbalanced datasets by integrating a cost-sensitive loss function, a nonlinear mapping strategy, and the SMOTE technique for oversampling; the approach achieves an accuracy of 96.27% and an AUC of 73.60%, outperforming other state-of-the-art models, including BAS, RUSBoost, and SVM-FK. The ESOA demonstrated strong convergence, computational efficiency, and superior generalizability across years of test data. However, while it excelled in accuracy and precision, it was slightly behind in the NDCG and required careful hyperparameter tuning. Future work includes refining FS and integrating deeper cost-sensitive learning to further increase real-world applicability [23]. Ranganatha et al. (2025) proposed an integrated framework for fraud detection FD in mobile transactions by combining bidirectional 3D quasi-recurrent neural networks Bi(3DQRNN) with blockchain-based proof of voting (PoV) consensus mechanisms. Their approach balances the data via self-adaptive synthetic oversampling (SASOS). They introduced an EAGTO algorithm to optimize the DL model's performance. The model achieves High detection accuracy of 97.1%, outperforming existing methods such as CNN-LSTM and ensemble models, with significantly reduced mining time and improved precision and specificity. The framework is robust even under simulated double-spending attacks. While it shows strong potential the limitations include reliance on Bitcoin-centric data and vulnerability to insider threats. Future directions suggest expanding to broader datasets and incorporating biometric or two-factor authentication for enhanced security [24].

Ensemble approaches have also gained prominence. El Bazi et al. (2024) proposed OptiStack, a robust stacking ensemble model for CCFD, which integrates XGBoost, CatBoost, and LightGBM as base learners with LR as the meta-learner. The framework incorporates Bayesian hyperparameter optimization via Optuna to fine-tune model parameters and SMOTE for class imbalance Handling; their model achieved an impressive ROC AUC score of 0.9887, with a recall of 0.86 and an F1 score of 0.87, reflecting strong sensitivity and balanced performance. Compared with the UAAD-FDNet model by Jiang et al. (2023), OptiStack outperformed in terms of recall, F1, and AUC but slightly underperformed in precision; the authors admit that precision still needs improvement and suggest future directions, such as developing more advanced hybrid models, incorporating behavioral features, and exploring real-time deployment strategies to further increase the effectiveness of fraud detection [25]. Additionally, Cherif et al. (2024) proposed an encoder-decoder graph neural network (GNN) model for detecting CCFs by modeling transactions as a Heterogeneous graph between customers and merchants. The study incorporates a rich feature engineering process, including spatial and temporal behavioral patterns such as transaction time, frequency, and geographical distance, to increase FD accuracy. Using a synthetic yet realistic large-scale dataset Sparkov, the model achieved 0.82 precision, 0.92 recall, 0.86 F1 score, and an AUROC of 0.92,

outperforming traditional ML and autoencoder-based methods. The approach leverages cosine similarity in the decoder for link prediction between graph nodes. The authors emphasize the effectiveness of graph neural networks (GNNs) in capturing local and global dependencies in transaction data. However, real-time implementation may face challenges in acquiring geolocation data. These authors suggest that future work should focus on scaling the model to massive graphs via distributed frameworks and advanced sampling techniques [26]. In [22], Tonui and Kibet (2025) evaluated the effectiveness of DL models, specifically convolutional neural networks (CNNs), long short-term memory (LSTM), and autoencoders in detecting CCFs. Using a Highly imbalanced dataset of 284,807 transactions, only 0.172% fraudulent, they applied SMOTE for resampling and MinMaxScaler for feature normalization. The CNN and LSTM models perform better by achieving 99.00% accuracy and High AUC scores, effectively identifying complex fraud patterns. However, the autoencoder showed limited effectiveness, with a low precision of 3.09%, underscoring the challenges of unsupervised methods in imbalanced datasets. XGBoost emerged as the best traditional model with 99.93% accuracy and an F1 score of 0.82. Limitations include the dependence on a single dataset and underexplored ensemble DL models. Future work suggests incorporating federated learning, diverse datasets with extended features, and feedback loops for adaptability to evolving fraud behaviors.

Several works have focused on enhancing interpretability and adaptability. Damanik and Liu (2025) introduced an advanced fraud detection framework that combines K-means SMOTEENN for data balancing and a stacking ensemble model composed of DT and random forest (RF) classifiers. Their method addresses class imbalance and overfitting, leveraging feature construction and resampling to increase model performance. Using the synthetic PaySim dataset, the proposed approach achieved 100% accuracy, 0.92 F1 score, 0.95 precision, 0.88 recall, 0.96 AUPRC, and a perfect 1.00 ROC-AUC, significantly outperforming traditional models such as XGBoost and LR. The inclusion of explainable AI via LIME provided interpretability by explaining the influence of individual features across base and meta-classifiers. Although highly effective, the model's computational cost, which requires up to 10 h of training on large datasets, poses deployment challenges. Future research should focus on enhancing scalability through dimensionality reduction, incremental learning, or optimizing ensemble training efficiency [27].

In [28], Alarfaj and Shahzadi (2025) presented an integrated framework for real-time CCFD in banking systems via GNNs and autoencoders. Their model leverages a lambda architecture that combines batch and speed layers to process historical and streaming data. The study uses transaction data from two Pakistani banks (Meezan and UBL), with preprocessing performed via min/max scaling to address severe class imbalance. The GNN component models transaction behaviors via a temporal graph and gated temporal attention network (GTAN), whereas the autoencoder identifies anomalies through reconstruction error. The results achieved classification accuracies of approximately 99.75% and 99.76% for the test and training datasets, respectively. The GTAN model outperformed the other models even with the smallest amount of labeled data; however, the study highlights some computational latency and scalability challenges, highlighting the need for further optimization techniques to support large-scale environments. Future directions include enhancing real-time responsiveness and integrating multimodal FD

systems for broader applicability. The authors of (2024) proposed an integrated multi-stage ensemble machine learning IMEML model to detect fraudulent credit card transactions. The model strategically combines the ensemble classifiers EIC, EBC, and EMC with advanced resampling techniques such as RUS, cluster centroids, and instance hardness threshold (IHT) variants to overcome class imbalance. The best performing configuration IHT + XGBoost within EIBMC achieved 99.94% accuracy, 99.91% precision, 99.14 recall, 99.52% Fi-score, and perfect 100% AUC, outperforming other recent models in the field. The model is computationally efficient and reliable, with a prediction time as low as 0.0749. Limitations include dependency on the quality of training data and evolving fraud patterns. Future work should explore deep feature engineering, alternative balancing techniques, and real-time data integration to improve adaptability and generalizability [29]. Iqbal and Amin (2024) presented a robust framework of time series forecasting and anomaly detection via DL with specific CCFD applications. The study evaluates multiple DL architectures, including LSTM, Autoencoders, LSTM-Autoencoder, Transformers, GANs, Deep-LSTM/CNN Hybrids, and Ensemble models. Their ensemble, LSTM-Autoencoder, and GAN models achieved the Highest results, with the LSTM-Autoencoder reaching 99.95% accuracy, 0.9861 F1 score, and 0.9918 AUC-ROC on the real dataset. They also introduce a novel statistical method for FS that maintains feature semantics while improving classification efficiency. The authors emphasize the potential of these models for real-time FD and stress the need for further research into scalability, interpretability, and multimodal integration across domains such as finance, healthcare, and cybersecurity [30].

Our comprehensive literature review focused on recent CCFD studies that apply optimization algorithms. We prioritized research that presents practical approaches for detecting fraudulent transactions and identifying fraudsters. Table 1 summarizes recent advancements in CCFD, highlighting studies utilizing various techniques, including ML, DL, and MHTs, and the contributions of each study, such as the proposed model, dataset, results, strengths, limitations, and future research directions. These insights underscore the evolving nature of CCFD and emphasize the importance of continued research to develop more effective, scalable, and adaptable detection systems.

Despite the remarkable progress in CCFD using hybrid ML/DL models and MHTs, existing approaches still face notable limitations in terms of FS efficiency, real-time scalability, and balancing exploration/exploitation trade-offs. Many models suffer from high computational costs, limited adaptability to evolving fraud behaviors, or reliance on synthetic datasets, which constrains their real-world applicability. While several studies integrate optimization algorithms, they often lack interpretability or dynamic adaptability during feature selection. The proposed BGVOA-LS algorithm addresses these gaps by combining the bioinspired GVOA algorithm with an LS strategy to enhance convergence and refine selected features. This is achieved by transforming continuous optimization into binary feature selection and incorporating an exploration-quality enhancer, the random repositioning of roaming agents. As a result, the BGVOA-LS achieves a robust balance between classification accuracy and computational efficiency. This enables the system to adapt to high-dimensional, imbalanced datasets, supporting scalable, interpretable, and real-time fraud detection systems.

Table 1 Recent studies in CCFD using optimization algorithms

Refs.	Used Method	Result	Dataset	Pro.	Cons.	Research gap	Future work
[31]	CSO + DCNN	Accuracy: 98.20% (Credit), 99.77% (Mortgage), 95.23% (Insurance)	Kaggle datasets (Credit, Mortgage, Insurance)	Improves fraud detection with high accuracy and lower errors (MAE, MSE)	CSO may face dispersion in long iterations	Stability issue of CSO over long iterations; evaluation concentrated on public datasets	Expand to sensitive datasets; apply ensemble preprocessing
[32]	OCSODL-CCFD	Acc: 99.97%, MCC: 91.62% Outperforms Competitors	Kaggle Credit Card Transactions	Reduced Complexity, Enhanced Classification, Superior Performance	Premature Convergence Possible	Susceptibility to premature convergence; limited use of clustering/outlier-detection stages	Explore Clustering and Outlier Detection
[33]	SVDD with PSLPSO	Accuracy improved from 90 to 93%	ECC (European Credit Cardholders)	Improved accuracy using feature selection and optimization; good anomaly detection	Needs tuning of hyperparameters C and σ	Lack of adaptive hyperparameter tuning and dynamic learning-rate control	Use fuzzy logic in PSLPSO to dynamically adapt learning rate
[34]	SMOTE + BinHHO + KNN/ SVM	Accuracy: up to 99.95% (KNN), 96.28% (SVM)	DTS1 (Payments), DTS2 (Cards)	Highly accurate; feature reduction, robust optimization	Needs tuning, SVM less optimal than KNN	High sensitivity to hyperparameters and classifier choice; generalization beyond tested domains not demonstrated	Extend to blockchain apps, deep learning, and other FS methods
[35]	DeepConvNet with SGD, Adagrad, Adam, RMSprop	RMSprop: 99.93%, Adagrad: 81.7%, SGD: 78%, Adam: 76.2%	UCI Credit Card dataset	RMSprop yields the highest accuracy, fast convergence, handles mini-batch and gradient smoothing efficiently	Basic optimizers like SGD and Adam offer lower accuracy and slower convergence	Performance heavily dependent on optimizer selection; absence of advanced/adaptive optimization strategies	Explore optimization strategies using reinforcement learning
[36]	JNBO–SpinalNet	Accuracy: 89.10% FPR: 10.20% MAP: 89.82% Loss: 13.16%	Credit card fraud dataset (Kaggle, European users)	High accuracy, privacy-preserving, effective optimization	Needs better data balance handling, complex tuning process	Insufficient handling of class imbalance; tuning complexity hampers practicality	Use more normalization methods and distance metrics for feature selection
[37]	BBBO + SVM, k-NN, Xgb-tree	Up to 91% accuracy with 67% attribute reduction	Australian credit dataset (UCI repository)	Efficient feature selection; handles local optima; fewer attributes selected with high accuracy	Requires complex behavior modeling; relatively new and untested on some datasets	Limited validation across diverse datasets; complexity of behavior modeling	Apply BBBO on wider datasets, explore hybrid ML-optimization models

Table 1 (continued)

Refs.	Used Method	Result	Dataset	Pro.	Cons.	Research gap	Future work
[38]	ACO + SVM	Credit: 96.44%, Loan & Insurance: 94.35%; Precision ~ 0.98–0.99, F1 ~ 0.94–0.96	Kaggle: Credit, Loan, Insurance datasets	Higher accuracy & recall versus SVM alone; effective fraud detection	Sensitive to parameter tuning, requires optimization	Parameter sensitivity without adaptive control; lack of real-time/adaptive capability	Extend with real-time data & adaptive learning strategies
[39]	ACO + Naive Bayes	3.44% → 1.96% prediction error reduced	Banking, Iris, Wine, Mall datasets	Enhances classification with fewer features; interpretable	Manual tuning of feature weights required	Absence of automated/learned feature-weight assignment; limited comparison against alternative FS for AML	Automate weight assignment; compare with other FS methods for AML applications
[40]	ENKMRH (K-means + + + RILHHO + ELM)	81.39%, 88.68%, 93.80% accuracy (3 datasets)	Internet Credit, Online Lending, AML (real data)	High accuracy, low std dev, efficient subset selection	Initial tuning needed; optimization sensitive to params	Parameter sensitivity and initial tuning overhead; lack of privacy-aware feature-sharing mechanisms	Explore privacy-aware sharing & advanced feature selection
[41]	GNN + HHO/CSO	Best F1: 87.74% (GNN + HHO with TF-IDF + BERT)	Amazon Reviews Dataset	Metaheuristic tuning boosts GNN accuracy beyond classical models	CSO less consistent; hyperparameter sensitive	Instability of CSO-based tuning; limited exploration of alternative GNN architectures	Explore other GNNs (e.g., GraphSAGE, attention), new optimizations

The proposed methodology

CCFD presents an active challenge due to the evolving nature of fraudulent behavior. To address these issues, we propose a novel hybrid approach, BGVOA-LS. This section provides a structured overview of the BGVOA-LS framework proposed for CCFD, as shown in Fig. 1, with a detailed examination of its components. The framework begins by utilizing three benchmark CCFD datasets, each of which is subjected to class balancing via RUS to mitigate the inherent class imbalance challenge. The balanced datasets are then input into the BGVOA, which performs FS by identifying the most informative and minimal subset of features. These selected features are subsequently evaluated via three classification algorithms (KNN, DT, and XGB-Tree) implemented in a wrapper-based evaluation strategy to ensure that the performance gains from BGVOA-LS are not tied to the strengths or weaknesses of any single model type but are instead robust across different decision-making mechanisms.

- DT: a symbolic, rule-based learner valued for its interpretability and decision transparency in regulated financial domains. DTs are lightweight at inference times and provide an explicit mapping between selected features and classification decisions.
- KNN: a nonparametric, instance-based learner whose performance is susceptible to feature quality and dimensionality. The KNN thus serves as a strong benchmark to assess the direct impact of feature subset optimization on distance-based decision boundaries.
- XGB-Tree: a state-of-the-art boosting ensemble with proven success in tabular, imbalanced datasets. Its capacity to model complex nonlinear relationships provides a high-performance baseline for evaluating BGVOA-LS under competitive conditions.

Other well-known used models, such as SVM and RF, were considered but excluded to avoid methodological redundancy or excessive computational cost. SVMs can be computationally intensive on large-scale datasets, especially when kernel tuning is needed,

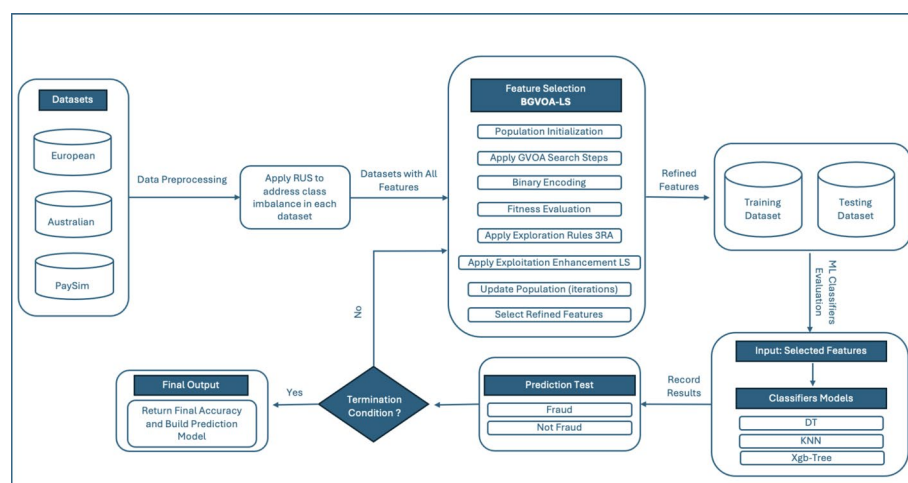


Fig. 1 Proposed framework

which could confound the evaluation of FS efficiency. Random forest (RF) offers a bagging-based ensemble approach, but its methodological similarity to XGB-tree and generally lower performance in recent fraud detection benchmarks make it less informative for diversity testing.

Advanced gradient boosting variants such as LightGBM and CatBoost, along with deep learning models, are acknowledged as promising for future work. However, they typically require more extensive hyperparameter tuning and hardware resources, and their inclusion increases the experimental complexity without expanding the diversity of the learning paradigms tested in this study.

By deliberately combining a transparent model DT, a distance-based learner KNN, and a high-performance gradient boosting method XGB-tree, we ensure that BGVOA-LS is evaluated under fundamentally different inductive biases. This provides more substantial evidence of the classifier-agnostic utility of the framework and supports its potential applicability in various operational contexts.

Data training and testing phases are applied for each classifier to assess its predictive performance. The resulting performance metrics are recorded and checked against predefined termination criteria. If the solution meets the terminating condition, the selected features are considered optimal. Otherwise, the BGVOA is iteratively reapplied to further refine the feature subset. This loop continues until convergence on a feature set that achieves maximum classification accuracy with the minimum number of features. The methodology ensures a synergistic optimization of feature relevance, model accuracy and computational efficiency across all stages.

Handling imbalanced classes via RUS

One of the most critical challenges in CCFD is High-class imbalance, where fraudulent transactions make up a tiny portion of the dataset, often less than 1%; this imbalance causes ML models to lean heavily toward predicting the majority class of legitimate transactions, resulting in high accuracy but poor recall for fraud instances. Resampling techniques such as RUS, SMOTE, and ADASYN are commonly applied to address class imbalance issues [42]. The RUS was selected for this study because of its computational efficiency and effectiveness, and the process begins by identifying the imbalance, which is typically characterized by a Highly skewed distribution of legitimate and fraudulent transactions. RUS then randomly eliminates samples from the majority class until a predefined class ratio such as 1:3, 1:27, or 1:81 is achieved, resulting in a more balanced dataset [43]. This undersampling is applied only to the training data during cross-validation to ensure unbiased model evaluation. The resulting balanced dataset allows the model to learn from both classes more effectively and reduces the bias toward the majority class; however, RUS may discard some informative data. RUS provides a simpler and faster alternative to synthetic oversampling methods, unlike SMOTE and ADASYN, which generate synthetic examples and require extensive parameter tuning, RUS avoids added complexity and the risk of introducing noise, making it a practical and scalable solution for large CCFD datasets. Empirical results from recent studies affirm that RUS significantly enhances recall and model robustness under extreme imbalance scenarios, particularly when evaluated via threshold-independent metrics such as the area under the precision-recall curve (AUPRC) [44].

Formulation and behavioral modeling for the proposed GVOA

The GVOA emphasizes its simplicity and power as a novel swarm-based metaheuristic inspired by the foraging behavior of griffon vultures (GV), leveraging its ecological and social dynamics to solve complex optimization problems. The algorithm systematically integrates biological behaviors into computational procedures to balance exploration and exploitation throughout the search process; GVs are large, social scavengers with high visual acuity and group foraging behavior, enabling them to locate food over vast terrains efficiently. The GV utilizes strategies such as group scanning, following experienced individuals, and cautious scouting near carcasses; these strategies are translated into the steps of the GVOA, influencing its search dynamics [18].

Population initialization

At the outset, the GVOA generates a random set of candidate solutions, forming the initial population; each solution corresponds to a vulture's position in the multidimensional search space, as shown in Eq. (1).

$$X = \begin{bmatrix} GV_{1,1} & GV_{1,2} & \cdots & \cdots & GV_{1,dim} \\ GV_{2,1} & GV_{2,2} & \cdots & \cdots & GV_{2,dim} \\ \vdots & \ddots & & & \ddots \\ \vdots & \ddots & & & \ddots \\ GV_{pop,1} & GV_{pop,2} & \cdots & \cdots & GV_{pop,dim} \end{bmatrix} \quad (1)$$

In this equation, pop refers to the count of candidate solutions, whereas dim indicates the dimensional size of the problem. The term $GV_{pop,dim}$ represents the location of the pop th GV along the dim th dimension. This approach guarantees a balanced distribution of GV values across the entire search area. These locations are randomly generated via upper and lower bounds of the search space and a random number, as described in Eq. (2).

$$X_{ij} = lb + Rand_1 \times (ub - lb) \quad (2)$$

In Eq. (2), lb and ub denote the lower and upper boundaries of the search space, whereas $Rand$ represents a uniformly identified random number in the range $[0,1]$. This ensures that the initial population is well spread across the feasible region.

The GVOA population update within the search space progresses through four straightforward steps: Step 1 emphasize the following behavior; Step 2, group-based foraging; Step 3, vulture self-guided search; and Step 4, near-carcass exploration.

Step 1: Emphasize the following behavior.

This step captures the social foraging behavior observed in griffon vultures, where less experienced individuals follow those with better knowledge to occasionally benefit from following others, as they may possess partial information about resource availability gained through experience in highly productive areas, and according to the following behavior described in [45], less experienced GVs follow griffons with more experienced individuals, allowing all vultures to occasionally benefit from

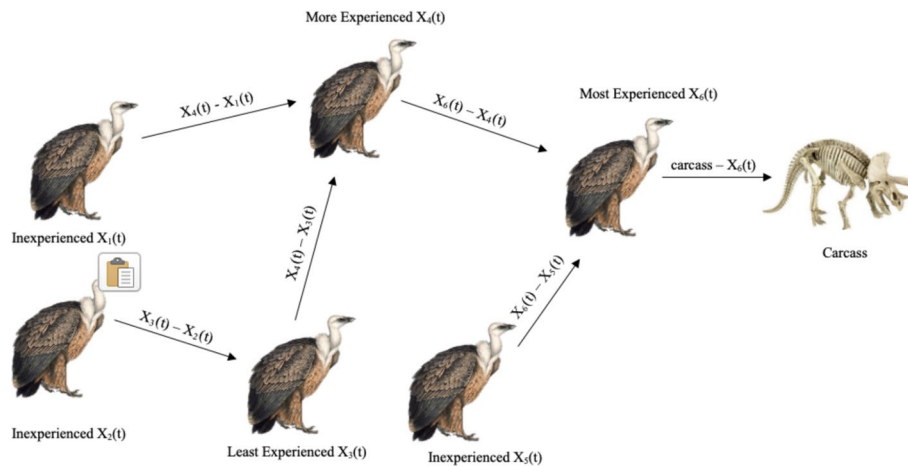


Fig. 2 GV following behavior

following others, as shown in Fig. 2. The figure illustrates the following behavior of griffon vultures, each possessing different levels of information. Although there is no formal information hierarchy among them, those with less information tend to follow those who are more knowledgeable. Consequently, each inexperienced vulture randomly selects an experienced vulture to follow. Similarly, experienced vultures benefit by following others and thus choose vultures with even greater experience. However, the most knowledgeable vulture has no more experienced individuals to follow, so it moves directly toward the carcass on the basis of its knowledge. Through this collective foraging behavior, GVs can efficiently cover extensive search areas, enhancing the global exploration effectiveness of the GVOA.

In this algorithmic context, the population of vultures (agents) is divided into experienced and inexperienced agents on the basis of their objective function values. For population size N , the $N/2$ agents with the highest objective values are categorized as inexperienced, whereas the remaining agents with lower objective values are labeled experienced. The one with the lowest objective value is designated the most experienced vulture. This classification framework is formally defined in Eq. (3). To model movement behavior, each inexperienced vulture ($X_i(t) \in \text{inexperiencedGV}$) selects a randomly experienced vulture (SelGV) to follow. In contrast, each experienced vulture attempts to follow another more experienced individual (one with a lower objective value) if such a vulture exists. This hierarchical following rule is governed by Eq. (4). The most informed vulture, having no superior counterpart, does not follow others but instead moves directly toward the carcass, symbolizing the global best solution found thus far.

Additionally, $b \neq \text{carcassIndex}$ ensures that the carcass itself is not selected as an experienced agent. The variable b is introduced in Eq. (5) to balance exploration and exploitation. Therefore, the variable enhances the ability of the GV to explore the search space across iterations through trial-and-error optimization. When the number of iterations is less than $\text{MaxIt}/3$, b is randomly selected from $[0,2]$ to emphasize exploitation. Conversely, when the iteration reaches or exceeds $\text{MaxIt}/3$, b is drawn from $[0,4]$ to encourage exploration. This adaptive adjustment maintains a balance between exploration and exploitation, particularly as Steps 3 and 4 of the proposed

BGVOA-LS algorithm emphasize exploitation in later stages. The position updating of the GV is mathematically modeled, where inexperienced vultures follow experienced ones. The most experienced vulture moves toward the carcass, as shown in Eq. (6) and Eq. (7); in this process, a new position is generated and accepted if it improves the objective function value, thereby supporting exploration in the early stages and enhancing exploitation during the later iterations.

$$\text{experiencedGV}, \text{inexperiencedGV} = \text{sort}(\text{fit}, \text{ascend}) [1 : N/2], [N/2 + 1 : N] \quad (3)$$

$$\text{SelGV} = \begin{cases} \text{rand}(\{X_b | b \in \text{experiencedGV}\}), & \text{if } X_i(t) \in \text{inexperiencedGV} \\ \text{rand}(\{X_b | \text{fit}_b < \text{fit}_i, b \neq \text{carcassIndex}\}), & i, b \in \text{experiencedGV}, \text{else} \end{cases} \quad (4)$$

$$b = 2 \times \text{rand} + \left(\text{it}^3 \frac{\text{MaxIt}}{3} \right) \times (2 \times \text{rand}). \quad (5)$$

$$X_{\text{FB}}^{P_1} = \begin{cases} X_i(t) + b \cdot (\text{carcassPos} - rN \times X_i(t)), & \text{SelGV} = \emptyset \\ X_i(t) + b \cdot \text{rand}(1, \text{dim}) \cdot (X_{\text{SelGV}} - rN \times X_i(t)), & \text{else} \end{cases} \quad (6)$$

$$X_i(t+1) = \begin{cases} X_{\text{FB}}^{P_1}, & \text{fit}_{\text{FB}}^{P_1} < \text{fit}_i \\ X_i(t), & \text{else} \end{cases} \quad (7)$$

In this context, *it* denotes the current iteration and *MaxIt* denotes the maximum allowable number of iterations. As defined in Eq. (5), *b* is randomly selected from [0,2] during the first *MaxIt*/3 iterations and from [0,4] thereafter, controlling the distance between an inexperienced GV and the more experienced one it follows. Where $X_{\text{FB}}^{P_1}$ represents the new position of a GV on the basis of the first following behavior step of the GVOA. If a more experienced vulture is available, it follows; otherwise, the GV moves toward the carcass.

Step 2: Group-based foraging.

In this step, GV forage collectively reduces the uncertainty of finding food by leveraging group dynamics; these GVs soar for long periods, scanning for carcasses but often delaying landing owing to the energetic cost and predation risk [46]. Leadership within the group is dynamic, shifting to the individual with the most updated information. Two main equations are proposed to model their group movement, which is controlled by the variable *cI* in Eq. (8). As outlined in Eq. (9), GV movement follows two conditions: if a random number of $\text{rand} < 0.2$, they move toward the carcass on the basis of the group's average position; otherwise, they adjust their position on the basis of both the group's mean and a randomly selected member. This approach reflects vultures' behavior of balancing cautious exploration with decisive movement toward food sources [47]. Suppose that a new position improves the objective function. In that case, it replaces the previous one, as defined in Eq. (10). The value of *cI* depends on a random variable *r*, drawn from a standard normal distribution if $r = 1$, or from a uniform [0,1] distribution if $r = 0$. The updated position $X_{\text{FG}}^{P_2}$ accounts for the group foraging phase involving both the carcass location and randomly selected

group members X_{rGM} . The group sizes vary, ranging from 9 to 60 individuals. Thus, the variable m is selected between 9 and the total population size N .

$$c1 = r \times \text{randn} + (1 - r) \times \text{rand} \quad (8)$$

$$X_{GF}^{P_2} = \begin{cases} X_i(t) + c1 \cdot \left(\text{carcassPos} - \frac{1}{m} \times \sum_{n=1}^m X^n(t) \right), \text{rand} < 0.2 \\ X_i(t) + c1 \cdot \left(X_{rGM}(t) - \frac{1}{m} \times \sum_{n=1}^m X^n(t) \right), \text{else} \end{cases} \quad (9)$$

$$X_i(t+1) = \begin{cases} X_{GF}^{P_2}, \text{fit}_{GF}^{P_2} < \text{fit}_i \\ X_i(t), \text{else} \end{cases} \quad (10)$$

Step 3: Vulture self-guided search.

GVs utilize both social and individual strategies to locate carcasses. They often follow experienced individuals while simultaneously using their exceptional vision to detect carcasses from long distances during flight [48]. In the proposed model, the parameter ω , introduced in Eq. (11), decreases linearly from 1 to 0 over time. Equation (12) independently defines the search area to represent the ability of the GV to find carcasses. Furthermore, Eq. (13) evaluates whether a randomly selected agent near a GV could indicate a better carcass on the basis of a lower objective function value. If the objective function value $\text{fit}_{VIS}^{P_3}$ at the new position $X_{VIS}^{P_3}$ is better than the current best-known *carcassScore* at *carcassPos*, the new position updates the carcass location, as detailed in Eq. (14). Therefore, $X_{VIS}^{P_3}$ represents the new location identified during the independent search step of the GVOA algorithm.

$$\omega = \left(1 - \frac{\text{it}}{\text{MaxIt}} \right) \quad (11)$$

$$\text{Search Range} : \begin{cases} \text{lb}_{VIS}^{\text{it}} = -r \times \omega + (1 - r) \times (\text{lb} \cdot \text{it}), \\ \text{ub}_{VIS}^{\text{it}} = r \times \omega + (1 - r) \times (\text{ub} \cdot \text{it}), \end{cases} \quad (12)$$

$$X_{VIS}^{P_3} = X_i(t) + \text{randn} \cdot (\text{ub}_{VIS}^{\text{it}} - \text{lb}_{VIS}^{\text{it}}) \quad (13)$$

$$\text{carcassPos} = \begin{cases} X_{VIS}^{P_3}, \text{fit}_{VIS}^{P_3} < \text{carcassScore} \\ \text{carcassPose}, \text{else} \end{cases} \quad (14)$$

Step 4: Near carcass exploration.

In the optimization context, the carcass metaphor represents an optimal or near-optimal solution that vultures attract but approach cautiously to balance risk and opportunity. To simulate this behavior, a control parameter a is introduced, which decreases nonlinearly from 1 to 0 as defined in Eq. (15). The squaring of $\left(1 - \frac{\text{it}}{\text{MaxIt}} \right)$ allows accelerated exploration in early stages and a gradual shift toward exploitation, enhancing convergence toward global optima. This nonlinear decay strategy outperforms simple linear reduction by maintaining an adaptive exploration/exploitation balance. Equation (16)

subsequently models localized exploration around the carcass by introducing a normally distributed perturbation, $a \times \text{randn}(1, \text{dim})$. If this exploration yields an improved objective function value, the new position supersedes the previous best position in Eq. (17). The $X_{\text{NCS}}^{P_4}$ term denotes the updated vulture position during the proximal carcass exploration step. The four behavioral steps of the GVOA within each iteration are defined by 17 equations and executed systematically for each agent by enabling a dynamic and adaptive search process. Upon reaching the maximum number of iterations, the algorithm returns the best-discovered candidate solution.

$$a = \omega^2 \quad (15)$$

$$X_{\text{NCS}}^{P_4} = \text{carcassPos} + \text{randn}(1, \text{dim}) \cdot a \quad (16)$$

$$\text{carcassPos} = \begin{cases} X_{\text{NCS}}^{P_4}, & \text{fit}_{\text{NCS}}^{P_4} < \text{carcassScore} \\ \text{carcassPose}, & \text{else} \end{cases} \quad (17)$$

Binary mapping of the GVOA positions in the FS

Reducing the number of features in a dataset without hindering model performance is a key part of FS. It helps focus on the most essential features and eliminates those that might negatively affect accuracy. The FS process can make the optimization algorithm work more efficiently and improve performance. When formulating FS as a binary optimization problem, the solution space must be represented via binary values [0,1], where 1 indicates selection and 0 indicates exclusion of a feature. However, the GVOA algorithm operates in a continuous search space and is inherently unsuitable for direct binary feature selection tasks. To bridge this gap, a transformation mechanism is employed to convert continuous-valued positions into binary representations. Each candidate solution is represented as a one-dimensional binary vector, with its length corresponding to the number of features in the dataset. A feature is selected if the corresponding binary entry is 1; otherwise, it is omitted. The transformation from a continuous position X_i^g at iteration g into its binary form $(X_i^g)_{\text{binary}}$ is performed via a threshold-based approach defined in Eq. (18), where a random threshold $\text{thr}_{\text{rand}} \in [0, 1]$ determines the discretization criterion. The selection of thr_{rand} values is context dependent and influenced by the characteristics and demands of the underlying optimization problem. Through this binarization process, the GVOA algorithm can efficiently address binary feature selection tasks and ensure the retention of essential features while eliminating redundant features.

$$(X_i^g)_{\text{binary}} = \begin{cases} 0 & \text{if } X_i^g < \text{thr}_{\text{rand}}, \\ 1 & \text{otherwise.} \end{cases} \quad (18)$$

Optimization of FS through fitness evaluation

Assessing FS effectiveness involves balancing between maximizing classification accuracy and minimizing the number of refined features; this balance is crucial to improving the efficiency and reliability of predictive model classification, and accuracy often

requires retaining more features, which can compromise model simplicity and efficiency. The optimal trade-off between accuracy and feature reduction is critical to ensure the performance and computational efficiency of the predictive model. In Eq. (19), the fitness function reflects this balance by integrating the classification accuracy and SF proportion. This formulation $(1 - \text{accuracy})$ corresponds to the classification error rate and D denotes the total number of features in the original dataset. d^* represents the number of features selected. Two weighting coefficients, ω_1 and ω_2 are crucial for handling the relative importance of classification performance and feature set size to achieve a balanced evaluation model solution. ω_1 is selected from the interval range $[0, 1]$, and ω_2 is assigned as $\omega_2 = 1 - \omega_1$. This approach ensures that solutions are evaluated not only on the basis of their predictive capability but also on their efficiency in reducing feature dimensionality.

$$\text{Fit} = \omega_1 \times (1 - \text{accuracy}) + \omega_2 \times \frac{|d^*|}{|D|} \quad (19)$$

Enhancing exploration quality via random repositioning agents (3RAs)

While exploring the search space in the context of the GVOA, agents may occasionally exceed the boundaries of the feature space, particularly given the high dimensionality of the FS problem. To address this, an effective boundary-handling or clipping strategy becomes crucial to prevent wandering individuals from moving outside the defined search space.

The studies have acknowledged the importance of such an enhancement step [49]; the techniques used to reintroduce out-of-bounds agents into the search space, whether by clamping positions to the boundary limits, randomly repositioning agents within the feasible space, or employing other strategies, are often not explicitly detailed, using random repositioning for agents that stray beyond the search boundaries. This approach, inspired by the inherent stochasticity of metaheuristic algorithms, is expected to improve search effectiveness. Accordingly, we have incorporated this strategy as an enhancement step aimed at mitigating the erratic behavior of agents and reducing the cycling issue often observed in the original GVOA. The pseudocode for the proposed 3RA strategy is provided in Algorithm 1.

Algorithm 1 3RA Technique.

```

 $[x_{\min}, x_{\max}]$  - continuous search domain, in our state,
 $[-1, 1]$ 
 $x_i^t$  - the current position of  $x$ 
For dimension  $j \in \{1, 2, \dots, D\}$  do
    If  $x_{ij}^t < x_{\min} \vee x_{ij}^t > x_{\max}$  then
         $x_{ij}^t \leftarrow U(x_{\min}, x_{\max})$ ;
    end if
end for
return  $x_i^t$ ;

```

Enhancing exploitation quality via LS

The LS is integrated into the BGVOA algorithm, as detailed in Algorithm 3. LS is applied at the end of each iteration $t + 1$ to refine the currently obtained best solution x_{best}^{t+1} , aiming to

remove any remaining irrelevant features. At the beginning of LS, a temporary copy of x_{best}^{t+1} is stored in a variable named *Temp*. The algorithm then proceeds for a predefined number of local search iterations, denoted as *LT*. In each local iteration *Lt*, four features' *rand_feat* are randomly selected from *Temp*, and their binary values are flipped such as switching between selected and deselected states. The modified *Temp* is then evaluated via the fitness function $f(\text{Temp})$. If the new fitness value improves upon that of the original x_{best}^{t+1} , the best solution is updated to match *Temp*. Otherwise, both *Temp* and the original best solution remain unchanged.

To evaluate the effectiveness of SE, the proposed improved BGVOA employs a wrapper-based approach via three classification techniques: KNN [50], Xgb-tree [51], and DT [52]. During each iteration, the selected subset of features produced by the BGVOA is used to train and test the dataset using each classifier individually. The FS process is represented in binary format, where a value of 1 indicates a selected feature and 0 indicates a deselected feature.

The overall workflow of the proposed BGVOA, including the integration of LS for local search, is illustrated in Algorithm 2.

Algorithm 2 BGVOA-LS Algorithm.

```

LT - maximum number of local iterations
 $x_{\text{best}}^{t+1}$  - the best position at the end of the BGVOA current iteration  $t + 1$ 
Temp  $\leftarrow x_{\text{best}}^{t+1}$ ;
Lt  $\leftarrow 0$ ;
While Lt < LT do
    rand_feat  $\leftarrow$  four features randomly selected from Temp;
    For feature  $a \in \text{rand\_feat}$  do  $\triangleright a \in \{0,1\}$ , where 1 means a feature is in while 0 means a feature is out
         $a \leftarrow \neg a$ ;
    end for
     $f(\text{Temp}) \leftarrow$  Calculate the fitness value of Temp;
    if  $f(\text{Temp}) < f(x_{\text{best}}^{t+1})$  then
         $x_{\text{best}}^{t+1} \leftarrow \text{Temp}$ ;
         $x_{\text{best}} \leftarrow x_{\text{best}}^{t+1}$ ;
         $f_g \leftarrow f(x_{\text{best}})$ ;
    end if
    Lt  $\leftarrow \text{Lt} + 1$ 
end while
return  $x_{\text{best}}, f_g$ ;

```

Algorithm design and workflow of the BGVOA-LS framework

The pseudocode of the proposed BGVOA-LS framework is presented in detail in Algorithm 3, Highlighting its core components. These key stages include dataset balancing, initial position generation, and position updates on the basis of the previously described GVOA strategies. The framework further incorporates the conversion of continuous positions into binary representations, the evaluation of fitness via a designed fitness function, and the enhancement of exploration capabilities via the integration of an LS technique. Together, these stages create a robust and efficient structure for the BGVOA-LS optimization process that is described in Algorithm 3.

Algorithm 3 Pseudocode of the BGVOA-LS Algorithm.

Input:
 N – Number of Vultures
 $MaxIt$ – Maximum number of iterations
 dim – Problem's dimensional space
 lb – lower bound of decision variable
 ub – upper bound of decision variable
 ω = Balance between exploration and exploitation through iterations
 $C1$ = Random variable controlling the distance of the target position
 b = Variable to explore search space
 a = Control intensity of the local search

Output:
 X_{best} – Best position found
 $fit(X_{best})$ – Best fitness function value found (should be minimized)

1: **Start**
2: Applying the RUS method to handle class imbalance data
3: Assign random positions to the initial population of N via Eq. (1) and Eq. (2);
4: For every initial population's position, calculate fitness function fit ;
5: Sort the positions in ascending order according to their calculated fitness function fit ;
6: Find the initial best position X_{best}^0 of the lowest fitness function fit (X_{best}^0) across all initial population positions;
7: $It \leftarrow 0$;
8: **While** $It < MaxIt$ **do**
9: **for** each vulture $i = 1$ to N **do**
10: Classify vultures: Experienced, Unexperienced $\leftarrow$ sort (fit , ascending) via Eq. (3)
11: Select a leader: $SeIGV \leftarrow rand(\{X_b | experienced\})$ via Eq. (4)
12: Update b for the move range via Eq. (5)
13: Update new position $X_{FB}^{P_1}$ via Eq. (6)
14: **if** $X_{FB}^{P_1}, fit_{FB}^{P_1} < fit_i$
15: Accept a new position via Eq. (7)
16: **end if**
17: Calculate $C1$ via random or uniform values via Eq. (8)
18: Apply group foraging update $X_{GF}^{P_2}$ via Eq. (9)
19: **if** $X_{GF}^{P_2}, fit_{GF}^{P_2} < fit_i$ **then**
20: Accept a new position via Eq. (10)
21: **end if**
22: Calculate $\omega = (1 - it/MaxIt)$ via Eq. (11)
23: Define the search range: $\begin{cases} lb_{VIS}^{it} = -r \times \omega + (1 - r) \times (lb \cdot /it), \\ ub_{VIS}^{it} = r \times \omega + (1 - r) \times (ub \cdot /it), \end{cases}$ via Eq. (12)
24: Generate a new position $X_{VIS}^{P_3}$ via Eq. (13)
25: **if** $X_{VIS}^{P_3}, fit_{VIS}^{P_3} < carcassScore$ **then**
26: Update carcassPos via Eq. (14)
27: **end if**
28: Calculate $a = \omega^2$ via Eq. (15)
29: Generate local move $X_{NCS}^{P_4}$ via Eq. (16)
30: **if** $X_{NCS}^{P_4}, fit_{NCS}^{P_4} < carcassScore$ **then**
31: Update carcassPos via Eq. (17)
32: **end if**
33: **end for**
34: $It \leftarrow It + 1$
35: transform positions to binary values via Eq. (18)
36: Re-Sort the Vultures' positions in ascending order on the basis of their computed fit values.
37: Find the best position X_{best}^0 and its best fitness value fit (X_{best}^0) at the end of the current iteration $It + 1$; using Eq. (18)
38: As explained in subsections 3.5 and 3.6, use 3RA and LS to refine X_{best}^0
39: **end while**
40: **return** X_{best}^0 and fit (X_{best}^0)
41: **End**

Complexity analysis of the BGVOA-LS

To better understand the efficiency of the proposed BGVOA-LS, we analyze its computational complexity ($O(BGVOA - LS)$) by considering each of its main stages. It begins with population initialization followed by position upgrading, which incorporates both the GVOA movements and binary mapping. The process then proceeds with fitness evaluation, random repositioning through the 3RA mechanism and, finally, the application of the LS strategy. Together, these stages capture the essential components that drive the overall computational cost of the algorithm.

Let N denote the number of vultures (individuals), the maximum number of iterations represented by $MaxIt$, and dim indicates the problem's dimensional space.

- $O(\text{Population initialization}) = O(N)$
- $O(\text{Position upgrading (GVOA moves + binary mapping)}) = O(N \times MaxIt \times dim)$

- $O(\text{Fitness evaluation}) = O(N \times \text{MaxIt})$
- $O(\text{Random repositioning 3RA}) = O(N \times \text{MaxIt} \times \text{dim})$
- $O(\text{LS strategy}) = O(N \times \text{MaxIt})$
- $O(\text{Random repositioning 3RA} + \text{LS strategy}) = O(N \times \text{MaxIt} \times \text{dim}) + O(N \times \text{MaxIt}) = O(N \times \text{MaxIt} \times \text{dim})$

By combining the complexities of all individual stages, the total computational complexity of the BGVOA_LS algorithm can be determined as:

$$O(\text{BGVOA-LS}) = O(N) + O(N \times \text{MaxIt} \times \text{dim}) + O(N \times \text{MaxIt} \times \text{dim}) + O(N \times \text{MaxIt})$$

Therefore, $O(\text{BGVOA-LS}) = O(N \times \text{MaxIt} \times \text{dim})$

We acknowledge that the computational complexity of the proposed BGVOA-LS is greater than that of the baseline GVOA because of the integration of binary feature mapping, random repositioning (3RA) for exploration, and the LS strategy for exploitation. Although these additions increase the computational cost, they significantly enhance the balance between exploration and exploitation, leading to faster convergence and improved feature selection performance. Importantly, this complexity remains comparable to that of other wrapper-based metaheuristic feature selection methods, where fitness evaluation dominates runtime and can be alleviated through parallel or distributed computing. Thus, the added complexity is a reasonable trade-off for achieving better robustness, scalability, and selection quality.

Results and analysis

This section presents comprehensive results and discussions derived from thorough experimental evaluations of the proposed BGVOA-LS algorithm; the results encompass detailed comparisons highlighting the performance, robustness, and effectiveness of the BGVOA-LS method against established benchmark algorithms. The discussions critically interpret these outcomes and explain the underlying reasons for the observed performance enhancements. A rigorous statistical analysis employing the Wilcoxon test substantiates these findings to illustrate statistically significant improvements clearly. The tables and figures included in this section effectively summarize the comparative outcomes and emphasize the superior performance of BGVOA-LS across various evaluation metrics and classification scenarios. Sects. "[Benchmark dataset description](#)" and "[Parameter configuration](#)" present the benchmark datasets and the parameter settings for the MHTs, respectively. The employed performance measurements are presented in Sect. "[Evaluation measurements](#)", and the proposed BGVOA-LS algorithm performance measurements are presented in Sect. "[BGVOA-LS algorithm performance measures](#)". Finally, the proposed BGVOA-LS algorithm results and future directions are explained in Sect. "[Real-time integration feasibility](#)".

Benchmark dataset description

This study utilizes three prominent benchmark datasets for CCFD (European, Australian, and PaySim) datasets from distinct financial environments that offer diverse representations of transaction behaviors. These datasets are essential for evaluating fraud detection models, especially in managing the prevalent class imbalance issue.

European dataset: The European dataset, often called the CCFD dataset, is among the most widely used resources in FD research. It comprises 284,807 credit card transactions collected over 2 days from European cardholders, with fraudulent transactions accounting for only 0.172% of the data. All the numerical input features are transformed via principal component analysis (PCA) to protect user privacy. The dataset is publicly available on Kaggle [4].

Australian dataset: The Australian dataset, sourced from an Australian financial institution, contains anonymized features related to credit card transactions along with fraud labels. It is a commonly used benchmark in financial fraud detection research. This dataset is a severe class imbalance challenge, as fraudulent transactions are notably rare, which complicates practical model training. The dataset is available through the UCI Machine Learning Repository [6].

PaySim dataset: The PaySim dataset is a synthetic dataset designed to replicate real-world mobile money transactions and aims to simulate fraud patterns in mobile-based financial systems. It includes over 6.3 million transactions across various types, such as cash-in, cash-out, transfer, and payment, with clearly labeled fraudulent cases. Although widely used in research on FD in mobile payment environments, the dataset presents a significant class imbalance, as fraudulent transactions constitute a tiny fraction of the total data. The dataset is publicly available on Kaggle [5].

A comparative analysis of the three datasets across several dimensions is presented in Table 2, including region, period, number of transactions, number of features, anonymization, data type, fraud type, and fraud percentage. Each dataset introduces distinct opportunities and challenges for FD research. The European dataset is highly popular but is characterized by a significant class imbalance. While the Australian dataset offers critical insights into approval-based fraud detection, the PaySim dataset focuses on mobile money transactions to make it particularly valuable for exploring fraud in digital banking systems.

Parameter configuration

The proposed BGVOA-LS was evaluated against the original BGVOA and eleven recent binary MHTs: (BMOA) Binary Meerkat Optimization Algorithm [53], (BAVO) Binary African Vulture Optimization [54], (BAO) Binary Aquila Optimizer [55], (BSSA) Binary Salp Swarm Algorithm [56], (BASO) Binary Atom Search Optimization [57], (BHGSO)

Table 2 CCF dataset comparison

Feature	European dataset	Australian dataset	PaySim dataset
Region	Europe	Australia	(Mobile Money) Simulated
Time Span	2 Days	Confidential	Simulated Data
No. of Transactions	284,807	690	~6.3 M
No. of Features	30 (PCA Transformation)	14	10+
Anonymization	Yes (PCA Transformation)	Yes	No (Synthetic Data)
Data Type	CC Transactions	CC Approvals	Mobile Money Transactions
Fraud Types	CCF	CCF	Mobile Money Fraud
Fraud Percentage	0.172%	~44.49%	~0.1%

Binary Henry Gas Solubility Optimization [58], (BHHO) Binary Harris Hawks Optimization [59], (BSFO) Binary Sailfish Optimizer [60], (BBA) Binary Bat Algorithm [61], (BGOA) Binary Grasshopper Optimization Algorithm [62], and (BPSO) Binary Particle Swarm Optimization [63]. To ensure a rigorous and unbiased evaluation, the experimental protocol strictly separated the training, validation, and testing data. For each benchmark dataset, an initial split allocated 80% of the data to the training set and reserved the remaining 20% as a final holdout test set. This holdout set was completely isolated and used only once to report the final performance metrics of the best-performing model, thus preventing any information leakage. Tenfold cross-validation was conducted exclusively within the training set as part of the wrapper-based feature selection process. In each fold, BGVOA-LS trains the classifier (DT, KNN, or Xgb-tree) on 90% of the training data via candidate feature subsets, whereas the remaining 10% serves as an internal validation set to compute the fitness score. This ensured that feature selection was guided by performance on unseen data in each fold, providing a robust estimate of generalization and reducing the risk of overfitting. To eliminate model selection bias, the hyperparameters for all the ML classifiers were fixed to standard, predefined values (Appendix A, Table 14) and were not tuned during the study. All algorithms were executed with a population size of 10, a maximum of 100 iterations, and a continuous search domain of $[-1, 1]$ relative to dataset attribute sizes. Each experiment was implemented in Python on a high-performance server (Dual Intel Xeon Gold 5115 CPUs, 128 GB RAM, Windows Server 2019), ensuring consistent and reproducible evaluation across different feature selection methods.

Evaluation measurements

BGVOA-LS effectiveness with the previously mentioned ML classifiers should be assessed via standard metrics to ensure the validity of the proposed algorithm. Table 3 shows the metrics used in detail.

BGVOA-LS algorithm performance measures

This section presents the key outcomes for the suggested algorithm. Table 4 compares three classifiers, DT, KNN, and XGB-Tree, across the European, Australian, and PaySim datasets. For each classifier dataset pair, two classifier values are reported: (i) the mean classification accuracy and (ii) the mean number of selected features after applying the BGVOA-LS. These values are generated by running each classifier in two modes, standalone and combined with the BGVOA-LS. The growth rate indicates the percentage improvement in classification accuracy with BGVOA-LS compared with the standalone classifier, whereas the reduction rate indicates the percentage decrease in the number of selected features relative to the total number of features in the original dataset. The results demonstrate that integrating BGVOA-LS consistently improves accuracy and reduces feature dimensionality, enabling faster and more effective CCFD.

In the second part of the experimental result analysis, the proposed BGVOA-LS algorithm is compared with several well-known MHTs on the European, Australian, and PaySim datasets. As shown in Tables 5, 6, and 7, the BGVOA-LS algorithm and MHTs are assessed, evaluated, and compared on the basis of the average accuracy, precision, recall, F1 score, specificity, kappa, ROC_AUC, fitness, number of SE, MCC, CCI, and ICI

Table 3 BGVOA-LS evaluation metrics

Metrics	Interpretation and calculations	Purpose
Accuracy	Measures all the proportion of correctly classified instances for both fraud and legitimate Where: True Positive (TP): Fraudulent transactions correctly identified True Negative (TN): Legitimate transactions correctly classified False Positive (FP): Legitimate transactions incorrectly classified as fraud False Negative (FN): Fraudulent transactions incorrectly classified as legitimate $ACC = \frac{TP+TN}{TP+TN+FP+FN}$	Assess the overall correctness of the classification to achieve higher accuracy to enhance model's performance
Precision (P) [64]	Measures the proportion of the transactions that correctly identified as fraud among all predicted frauds $P = \frac{TP}{TP+FP}$	Focuses on minimizing false positives where legitimate transactions wrongly flagged as fraud
Recall (R)[65]	Measures the proportion of the actual frauds that are correctly detected $R = \frac{TP}{TP+FN}$	Ensures that the fraud scenarios are captured, and minimizing undetected fraud
F1-Measure [66]	Measure the proportion harmonic mean of precision and recall, providing a single metric that balances both $F1 = 2 \times \frac{P \times R}{P+R}$	High F1 measure ensures the balance between precision and recall into a single comprehensive metric, useful for imbalanced data
Specificity (SPC) [67]	Measures the proportion of the legitimate transactions that correctly identified $SPC = \frac{TN}{TN+FP}$	High specificity reduces false positives by correctly classifying nonfraudulent transaction
Cohen's Kappa [68]	Evaluates the agreement between the predicted and the actual classifications, accounting for the chance agreement Where: P_o = Observed agreement & P_e = expected chance agreement $Kappa = \frac{P_o - P_e}{1 - P_e}$	Kappa value approximates to 1 indicates the strong agreement between the prediction and the actual classifications
ROC & AUC-ROC [69]	The ROC curve illustrates True Positive Rate (TPR) versus False Positive Rate (FPR), and the AUC-ROC score evaluates the model's ability to distinguish fraud from legitimate transactions	The higher AUC indicates superior ability to distinguish between fraudulent and legitimate transactions
Matthews Correlation Coefficient (MCC)[70]	A comprehensive metric considering TP, TN, FP and FN $MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$	The values near +1 indicates the higher classification. MCC is effective for evaluation the imbalanced data-sets performance
Fitness Function [71]	An evaluation metric that evaluates Model's accuracy and FS	Balances the maximization of the classification performance and minimization of the computational complexity
Number of Selected Features [72]	An evaluation metric that reflects the number of retained features after the FS using BGVOA-LS	Reducing features while maintaining accuracy improves model efficiency, speed, and interpretability
Correct Classification instances (CCI)[73]	Total number of correctly classified transactions $CCI = TP + TN$	Measures directly the model's success
Incorrect Classification Instances (ICI)[73]	Total number of misclassified transactions $ICI = FP + FN$	Measures directly the model's errors and weakness

Table 4 BGVOA-LS versus ML classifiers in terms of the mean (accuracy & SF) for CCFD

Dataset	Accuracy				Selected features		
	Classifiers	Classifier Value	BGVOA-LS	Growth Rate %	Classifier Value	BGVOA-LS	Reduction Rate %
European	DT	0.7614	0.9472	24.40%	30	14.7667	50.70%
Australian		0.7319	0.9118	24.57%	14	8.7	37.85%
PaySim		0.8652	0.9985	15.41%	9	3	66.60%
European	KNN	0.6548	0.9474	44.68%	30	11.5667	61.44%
Australian		0.8116	0.9039	11.37%	14	6.2333	55.47%
PaySim		0.7097	0.9914	39.70%	9	2.9667	67%
European	Xgb-Tree	0.9239	0.9601	3.91%	30	13.6	54.66%
Australian		0.8623	0.9135	5.93%	14	6.5667	53%
PaySim		0.9985	0.9985	0.00%	9	3	66.60%

The bold values highlight the performance of the proposed BGVOA-LS algorithm in terms of accuracy and feature selection

respectively; the bold results indicate the best values for each ML classifier DT, KNN, and Xgb-Tree in the tables.

According to Table 5, which compares BGVOA-LS with other optimization algorithms via the DT classifier, the proposed BGVOA-LS algorithm consistently ranks first across all three datasets (European, Australian, and PaySim). Specifically, BGVOA-LS achieved the highest mean accuracy (0.9472, 0.9118, and 0.9985, respectively), F1 score, and other key metrics, including the MCC and kappa. Even without the LS component, BGVOA ranks second, highlighting the critical enhancement that LS introduces. The other top algorithms, such as the BMOA and BAVO, weakened behind, especially with respect to metrics such as the recall and MCC, confirming the superior predictive strength of the BGVOA-LS. From Table 6, which uses the KNN classifier, BGVOA-LS again demonstrates dominant performance. It ranked highest in terms of accuracy (0.9474 for the European and 0.9039 for the Australian) datasets, precision, and F1 score in most cases. While its closest competitors, such as BAO and BSSA, demonstrated competitive metrics in precision and specificity, they consistently fell short in overall consistency and feature size selection. The BGVOA without LS performed notably but was still outperformed by the complete BGVOA-LS algorithm. BHGSO showed unusually high variance and instability. This further supports the robustness and generalizability of the BGVOA-LS. Table 7, which uses the Xgb-tree classifier, once again positions BGVOA-LS at the top. It achieved superior mean accuracy scores (0.9601 for the European, 0.9135 for the Australian, and 0.9985 for the PaySim) datasets and maintained high precision and recall. Among the other algorithms, the BMOA yields competitive recall and ROC scores but fails to match the BGVOA-LS in terms of overall F1 and MCC values. BGVOA-LS selects fewer features while achieving better classification metrics, confirming its effectiveness for FS.

These results validate that the proposed model not only enhances classification performance across diverse datasets and classifiers but also holds strong potential as a comprehensive feature selection tool that enhances exploration and exploitation.

Figures 3, 4, and 5 imply that the selected attribute of the proposed algorithm achieves the highest performance compared with the other selected optimization

Table 5 BGVOA-LS versus optimization algorithms for CCFD via the DT classifier

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPSP
European	Mean-Accuracy	0.9472	0.9423	0.9266	0.9377	0.9391	0.9382	0.9293	0.9284	0.9357	0.932	0.9274	0.9362	0.9389
	SD-Accuracy	0.0045	0.0052	0.0086	0.007	0.0059	0.0059	0.0057	0.0053	0.0076	0.0048	0.0059	0.0055	0.0053
Australian	Mean-Accuracy	0.9118	0.908	0.8729	0.9019	0.899	0.899	0.8688	0.8836	0.8882	0.8918	0.8814	0.9019	0.8923
	SD-Accuracy	0.0027	0.007	0.0109	0.0083	0.0114	0.0104	0.0145	0.0112	0.0104	0.012	0.0149	0.01	0.0109
PaySim	Mean-Accuracy	0.9985	0.9985	0.9975	0.9985	0.9984	0.9985	0.9977	0.998	0.9984	0.9985	0.998	0.9984	0.9983
	SD-Accuracy	0.0000	0.0000	0.0015	0.0000	0.0003	0.0000	0.0012	0.0006	0.0002	0.0000	0.0007	0.0002	0.0004
European	Mean-Precision	0.9783	0.974	0.9695	0.9729	0.9708	0.9753	0.9502	0.9701	0.97	0.9661	0.9659	0.9766	0.9758
	SD-Precision	0.0158	0.0103	0.0203	0.0165	0.0145	0.0199	0.0511	0.0176	0.0145	0.018	0.0191	0.0186	0.0174
Australian	Mean-Precision	0.8848	0.878	0.8489	0.8755	0.877	0.8791	0.7012	0.8531	0.8707	0.8747	0.8587	0.8746	0.8801
	SD-Precision	0.0102	0.0137	0.0549	0.0241	0.0334	0.0248	0.1921	0.0389	0.0383	0.0336	0.0522	0.0267	0.0269
PaySim	Mean-Precision	1.0000	1.0000	0.9993	1.0000	0.9999	1.0000	0.9986	0.9994	0.9999	1.0000	0.9994	1	0.9998
	SD-Precision	0.0000	0.0000	0.0006	0.0000	0.0003	0.0000	0.1009	0.0008	0.0003	0.0000	0.0011	0.0002	0.0005
European	Mean-Recall	0.9146	0.9085	0.881	0.9003	0.9051	0.8993	0.8973	0.884	0.899	0.8952	0.8861	0.8939	0.9
	SD-Recall	0.0157	0.0136	0.0265	0.0166	0.0149	0.02	0.0212	0.0191	0.0203	0.0169	0.0219	0.0217	0.0145
Australian	Mean-Recall	0.8758	0.8725	0.8085	0.8582	0.8484	0.8444	0.8203	0.832	0.8235	0.8288	0.8209	0.8595	0.8222
	SD-Recall	0.0178	0.0181	0.0632	0.0319	0.0395	0.0385	0.0608	0.0429	0.0464	0.0447	0.0559	0.0328	0.0402
PaySim	Mean-Recall	0.9969	0.9969	0.9957	0.9969	0.9969	0.9969	0.9962	0.9967	0.9969	0.9969	0.9966	0.9969	0.9968
	SD-Recall	0.0000	0.0000	0.0027	0.0000	0.0003	0.0000	0.0018	0.0006	0.0002	0.0000	0.0005	0.0002	0.0004
European	Mean-F1 Score	0.9452	0.94	0.9226	0.935	0.9366	0.9354	0.9266	0.9247	0.9329	0.929	0.9239	0.933	0.9361
	SD-F1 Score	0.0047	0.0056	0.0099	0.0074	0.0062	0.0063	0.006	0.006	0.0085	0.0051	0.0068	0.0063	0.0054
Australian	Mean-F1 Score	0.8801	0.8751	0.8242	0.866	0.8612	0.8605	0.8216	0.8407	0.8445	0.8496	0.8363	0.8661	0.8491
	SD-F1 Score	0.0053	0.0099	0.0163	0.012	0.0161	0.0165	0.0215	0.0151	0.0153	0.018	0.0201	0.0144	0.0173
PaySim	Mean-F1 Score	0.9985	0.9985	0.9975	0.9985	0.9984	0.9985	0.9977	0.998	0.9984	0.9985	0.998	0.9984	0.9983
	SD-F1 Score	0.0000	0.0000	0.0015	0.0000	0.0003	0.0000	0.0012	0.0006	0.0002	0.0000	0.0007	0.0002	0.0004
European	Mean-SPC	0.9795	0.9758	0.9717	0.9747	0.9727	0.9768	0.9609	0.9724	0.9721	0.9684	0.9684	0.9781	0.9774
	SD-SPC	0.0155	0.01	0.0198	0.0158	0.0143	0.0195	0.0218	0.0173	0.014	0.0178	0.0188	0.0183	0.017

Table 5 (continued)

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPPO
Australian	Mean-SPC	0.933	0.9287	0.9107	0.9276	0.9287	0.931	0.8973	0.9138	0.9261	0.9287	0.9169	0.9268	0.9333
	SD-SPC	0.0073	0.0096	0.0434	0.0188	0.0247	0.0178	0.0387	0.0301	0.0292	0.0258	0.0398	0.0196	0.0193
PaySim	Mean-SPC	1.0000	1.0000	0.9993	1.0000	0.9999	1.0000	0.9993	0.9994	0.9999	1.0000	0.9994	1	0.9998
	SD-SPC	0.0000	0.0000	0.0006	0.0000	0.0003	0.0000	0.0013	0.0008	0.0003	0.0000	0.0011	0.0002	0.0005
European	Mean-Kappa	0.8944	0.8846	0.8531	0.8754	0.8781	0.8764	0.8585	0.8568	0.8714	0.8639	0.8548	0.8724	0.8778
	SD-Kappa	0.0089	0.0103	0.0172	0.0141	0.0117	0.0118	0.0114	0.0106	0.0151	0.0097	0.0118	0.011	0.0106
Australian	Mean-Kappa	0.8104	0.8022	0.7251	0.7887	0.7819	0.7815	0.7182	0.7491	0.7574	0.7653	0.7436	0.7888	0.7656
	SD-Kappa	0.0068	0.0153	0.0222	0.018	0.0244	0.0236	0.0312	0.0233	0.0223	0.0263	0.0308	0.0217	0.0247
PaySim	Mean-Kappa	0.997	0.997	0.995	0.997	0.9968	0.997	0.9955	0.9961	0.9968	0.997	0.996	0.9969	0.9965
	SD-Kappa	0.0000	0.0000	0.003	0.0000	0.0006	0.0000	0.0025	0.0012	0.0005	0.0000	0.0015	0.0003	0.0008
European	Mean-ROC	0.9596	0.959	0.9495	0.954	0.9568	0.9567	0.9524	0.9465	0.9533	0.9523	0.9494	0.9534	0.9566
	SD-ROC	0.0092	0.0084	0.0132	0.013	0.0095	0.0089	0.0137	0.013	0.0123	0.0106	0.012	0.0121	0.0112
Australian	Mean-ROC	0.8104	0.8022	0.7251	0.7887	0.7819	0.7815	0.7182	0.7491	0.7574	0.7653	0.7436	0.7888	0.7656
	SD-ROC	0.0068	0.0153	0.0222	0.018	0.0244	0.0236	0.0312	0.0233	0.0223	0.0263	0.0308	0.0217	0.0247
PaySim	Mean-ROC	0.999	0.9989	0.9988	0.9989	0.9989	0.999	0.9987	0.999	0.999	0.999	0.9988	0.999	0.9988
	SD-ROC	0.0001	0.0001	0.0004	0.0001	0.0003	0.0001	0.0006	0.0004	0.0002	0.0001	0.0005	0.0001	0.0003
European	Mean-Fitness	0.0572	0.062	0.0779	0.0664	0.0649	0.066	0.075	0.0767	0.0686	0.0727	0.0766	0.0682	0.0655
	SD-Fitness	0.0044	0.005	0.0084	0.007	0.0057	0.0057	0.0053	0.0054	0.0073	0.0047	0.0057	0.0053	0.0052
Australian	Mean-Fitness	0.0935	0.097	0.1311	0.103	0.1054	0.1056	0.135	0.1211	0.1157	0.1127	0.123	0.1027	0.1119
	SD-Fitness	0.0022	0.0061	0.0103	0.0078	0.0108	0.0098	0.0138	0.0111	0.0094	0.0113	0.0145	0.0091	0.0104
PaySim	Mean-Fitness	0.0048	0.0048	0.0071	0.0048	0.0049	0.0048	0.0069	0.0063	0.005	0.0048	0.0058	0.0049	0.005
	SD-Fitness	0.0000	0.0000	0.0024	0.0000	0.0002	0.0000	0.0022	0.0011	0.0004	0.0000	0.0015	0.0002	0.0005
European	Mean-Feature Size	14.7667	14.6	15.5667	14.2667	13.7333	14.6667	14.9333	17.6333	14.9667	16.1667	14.3	15.2333	15
	SD-Feature Size	2.2164	2.7154	3.18	2.67	2.6196	2.0055	3.1298	2.1367	2.7384	2.3816	2.9905	2.4723	2.1602
Australian	Mean-Feature Size	8.7000	8.2667	7.5000	8.2667	7.6333	7.9333	7.2333	8.2333	6.9667	7.8333	7.7667	7.8333	7.3
	SD-Feature Size	0.9	1.5261	1.8574	1.2365	1.7604	1.6111	2.0605	1.3337	2.1211	1.655	1.4533	1.5723	1.7349

Table 5 (continued)

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPPO
PaySim	Mean-Feature Size	3.0000	3.0000	4.1667	3.0000	2.9667	3.0000	4.2333	3.9333	3.0667	3.0000	3.4333	3.0000	2.9667
European	SD-Feature Size	0.0000	0.0000	1.5510	0.0000	0.3145	0.0000	1.4067	0.9286	0.4422	0.0000	1.116	0	0.4819
	Mean-MCC	0.8966	0.8867	0.8573	0.8782	0.8804	0.8797	0.861	0.8606	0.8741	0.8667	0.8583	0.8761	0.8808
	SD-MCC	0.0089	0.0099	0.016	0.0141	0.0117	0.0117	0.0117	0.0106	0.0143	0.0099	0.0114	0.0105	0.0111
Australian	Mean-MCC	0.8106	0.8025	0.7298	0.7897	0.7836	0.7829	0.7215	0.7511	0.7602	0.7676	0.7473	0.7898	0.7677
	SD-MCC	0.0062	0.0151	0.0208	0.0173	0.0237	0.0226	0.031	0.0232	0.0214	0.0251	0.0294	0.0207	0.024
PaySim	Mean-MCC	0.997	0.997	0.995	0.997	0.9968	0.997	0.9955	0.9961	0.9968	0.997	0.996	0.9969	0.9965
	SD-MCC	0.0000	0.0000	0.0030	0.0000	0.0006	0.0000	0.0025	0.0012	0.0005	0.0000	0.0015	0.0003	0.0008
European	Mean-CCI	878	878	876	877	878	877	878	876	877	877	876	876	877
	SD-CCI	2.9477	2.1084	4.2797	2.8744	2.6407	3.721	4.0857	3.4301	3.0847	3.2836	3.8426	3.792	2.9403
Australian	Mean-CCI	602	602	601	602	601	601	602	601	600	600	601	602	599
	SD-CCI	1.5221	1.464	6.8557	3.0543	3.8621	3.2345	6.1519	4.5675	4.6953	4.209	5.9964	3.0811	3.4345
PaySim	Mean-CCI	14,772	14,772	14,771	14,772	14,772	14,772	14,771	14,772	14,772	14,772	14,772	14,772	14,772
	SD-CCI	0.0000	0.0000	4.2375	0.0000	0.0000	0.0000	3.2735	1.0160	0.3590	0.0000	1.4318	0	0.5385
European	Mean-ICI	105	105	107	106	105	106	105	107	106	106	107	107	106
	SD-ICI	2.9477	2.1084	4.2797	2.8744	2.6407	3.721	4.0857	3.4301	3.0847	3.2836	3.8426	3.792	2.9403
Australian	Mean-ICI	87	87	89	87	88	88	87	88	89	89	88	87	90
	SD-ICI	1.5221	1.464	6.8557	3.0543	3.8621	3.2345	6.1519	4.5675	4.6953	4.209	5.9964	3.0811	3.4345
PaySim	Mean-ICI	1654	1654	1654	1654	1654	1654	1654	1653	1653	1654	1653	1654	1653
	SD-ICI	0.0000	0.0000	4.2375	0.0000	0.0000	0.0000	3.2735	1.0160	0.3590	0.0000	1.4318	0	0.5385

The bold values indicate the highest values achieved by the algorithms

Table 6 BGVOA-LS versus optimization algorithms for CCFD via the KNN classifier

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BF50
European	Mean-Accuracy	0.9474	0.9447	0.9315	0.9474	0.9462	0.945	0.9308	0.9342	0.945	0.9372	0.9321	0.9418	0.9377
	SD-Accuracy	0.0033	0.0036	0.0064	0.0046	0.0036	0.0035	0.0062	0.0028	0.0047	0.0033	0.0048	0.0031	0.0035
Australian	Mean-Accuracy	0.9039	0.9029	0.8845	0.9019	0.9027	0.9014	0.878	0.8865	0.8986	0.8935	0.887	0.901	0.8944
	SD-Accuracy	0.0032	0.004	0.0079	0.0045	0.0036	0.0044	0.0075	0.0063	0.0059	0.0042	0.0091	0.0047	0.0055
PaySim	Mean-Accuracy	0.9914	0.9915	0.9902	0.9914	0.9915	0.9915	0.9907	0.9908	0.9913	0.9913	0.9904	0.9913	0.9909
	SD-Accuracy	0.0002	0	0.0018	0.0002	0	0	0.0006	0.0006	0.0004	0.0005	0.0011	0.0004	0.0005
European	Mean-Precision	0.9769	0.9722	0.9646	0.9748	0.9758	0.9743	0.8761	0.9675	0.974	0.9732	0.9687	0.9718	0.9691
	SD-Precision	0.0074	0.0064	0.0087	0.0076	0.0083	0.0066	0.1987	0.0068	0.0063	0.0065	0.0077	0.0072	0.008
Australian	Mean-Precision	0.9026	0.902	0.855	0.9016	0.9024	0.8997	0.7722	0.8679	0.8896	0.8805	0.863	0.8972	0.8814
	SD-Precision	0.0167	0.0206	0.0225	0.0206	0.019	0.0198	0.1514	0.0202	0.0232	0.0161	0.0264	0.0242	0.0183
PaySim	Mean-Precision	0.9865	0.9861	0.9874	0.9865	0.9861	0.9861	1.0731	0.9874	0.9874	0.9882	0.9909	0.9874	0.9909
	SD-Precision	0.0023	0	0.0061	0.0023	0	0	0.2185	0.0045	0.0038	0.0047	0.007	0.0038	0.006
European	Mean-Recall	0.916	0.915	0.8952	0.918	0.9146	0.9136	0.8925	0.898	0.9139	0.8986	0.8925	0.9095	0.9037
	SD-Recall	0.0097	0.0076	0.0134	0.0104	0.0093	0.0105	0.0101	0.0075	0.0094	0.0064	0.0101	0.0082	0.0086
Australian	Mean-Recall	0.8301	0.8281	0.8294	0.8255	0.8268	0.8261	0.8183	0.8183	0.8294	0.8242	0.8268	0.8281	0.8261
	SD-Recall	0.0155	0.0173	0.0254	0.0155	0.0161	0.0141	0.0361	0.0208	0.0184	0.0186	0.0254	0.018	0.015
PaySim	Mean-Recall	0.9964	0.9969	0.9931	0.9964	0.9969	0.9969	0.9925	0.9943	0.9953	0.9944	0.9899	0.9953	0.991
	SD-Recall	0.0027	0	0.006	0.0027	0	0	0.0066	0.005	0.0046	0.0057	0.0072	0.0046	0.0071
European	Mean-F1 Score	0.9454	0.9427	0.9285	0.9455	0.9442	0.9429	0.9277	0.9314	0.943	0.9344	0.929	0.9396	0.9352
	SD-F1 Score	0.0037	0.0038	0.007	0.005	0.0038	0.0039	0.0065	0.003	0.0051	0.0035	0.0052	0.0034	0.0037
Australian	Mean-F1 Score	0.8645	0.8631	0.8415	0.8615	0.8626	0.8611	0.832	0.8419	0.858	0.8511	0.8439	0.8608	0.8526
	SD-F1 Score	0.0044	0.0049	0.0107	0.0052	0.0044	0.005	0.0118	0.0086	0.0074	0.0065	0.0122	0.005	0.0071
PaySim	Mean-F1 Score	0.9914	0.9915	0.9902	0.9914	0.9915	0.9915	0.9907	0.9908	0.9913	0.9913	0.9904	0.9913	0.9909
	SD-F1 Score	0.0002	0	0.0018	0.0002	0	0	0.0007	0.0006	0.0004	0.0005	0.001	0.0004	0.0006
European	Mean-SPC	0.9785	0.9741	0.9673	0.9764	0.9774	0.9761	0.9687	0.97	0.9758	0.9754	0.9714	0.9737	0.9714
	SD-SPC	0.0073	0.0062	0.0085	0.0075	0.0081	0.0066	0.0109	0.0066	0.0062	0.0062	0.0074	0.0072	0.0079

Table 6 (continued)

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPSO
Australian	Mean-SPC	0.9471	0.9467	0.9169	0.9467	0.9471	0.9456	0.913	0.9264	0.9391	0.9341	0.9222	0.9437	0.9345
	SD-SPC	0.0105	0.0131	0.0175	0.0131	0.0121	0.0126	0.0217	0.0144	0.0149	0.0111	0.0187	0.0155	0.0119
PaySim	Mean-SPC	0.9865	0.9861	0.9873	0.9865	0.9861	0.9861	0.989	0.9873	0.9873	0.9882	0.9909	0.9874	0.9908
	SD-SPC	0.0023	0	0.0062	0.0023	0	0	0.0059	0.0045	0.0038	0.0047	0.0071	0.0038	0.0061
European	Mean-Kappa	0.8947	0.8893	0.8629	0.8947	0.8923	0.89	0.8615	0.8683	0.89	0.8744	0.8642	0.8835	0.8754
	SD-Kappa	0.0067	0.0071	0.0128	0.0093	0.0072	0.007	0.0124	0.0056	0.0095	0.0067	0.0096	0.0063	0.0069
Australian	Mean-Kappa	0.7902	0.7881	0.7507	0.7859	0.7875	0.7849	0.7364	0.7535	0.7793	0.7684	0.7554	0.7841	0.7705
	SD-Kappa	0.0067	0.0081	0.0167	0.0089	0.0072	0.0087	0.0165	0.0133	0.0121	0.0094	0.0191	0.009	0.0115
PaySim	Mean-Kappa	0.9829	0.983	0.9804	0.9829	0.983	0.983	0.9815	0.9816	0.9826	0.9825	0.9809	0.9827	0.9819
	SD-Kappa	0.0004	0	0.0037	0.0004	0	0	0.0012	0.0011	0.0008	0.0009	0.0021	0.0007	0.0011
European	Mean-ROC	0.9642	0.9642	0.955	0.963	0.9625	0.9627	0.9566	0.9573	0.9631	0.9608	0.9567	0.9598	0.96
	SD-ROC	0.0078	0.0077	0.0068	0.0075	0.0075	0.0067	0.0073	0.0067	0.0071	0.0068	0.0089	0.0077	0.007
Australian	Mean-ROC	0.9099	0.9096	0.9011	0.9084	0.9084	0.9083	0.8933	0.9	0.9083	0.9044	0.9011	0.9091	0.9068
	SD-ROC	0.0051	0.0064	0.0132	0.007	0.0062	0.0068	0.0139	0.0136	0.0078	0.0122	0.0139	0.0088	0.0102
PaySim	Mean-ROC	0.9945	0.9944	0.995	0.9945	0.9944	0.9944	0.9955	0.9954	0.9948	0.9949	0.9959	0.9948	0.9958
	SD-ROC	0.0004	0	0.0016	0.0004	0	0	0.001	0.0009	0.0008	0.001	0.0013	0.0008	0.0011
European	Mean-Fitness	0.056	0.0589	0.0725	0.0562	0.0569	0.0587	0.0733	0.0703	0.0583	0.0666	0.0717	0.062	0.066
	SD-Fitness	0.0032	0.0035	0.0063	0.0046	0.0037	0.0034	0.0064	0.0027	0.0048	0.0028	0.0047	0.0031	0.0034
Australian	Mean-Fitness	0.0996	0.1007	0.1196	0.1016	0.1007	0.102	0.1261	0.1178	0.1051	0.1103	0.1169	0.103	0.1095
	SD-Fitness	0.0033	0.0039	0.0077	0.0043	0.0033	0.0041	0.0072	0.0059	0.0058	0.0042	0.0087	0.0048	0.0057
PaySim	Mean-Fitness	0.0118	0.0118	0.0136	0.0118	0.0118	0.0118	0.013	0.0131	0.0118	0.0118	0.0131	0.0118	0.0121
	SD-Fitness	0	0	0.0025	0	0	0	0.0011	0.0008	0.0002	0.0001	0.0013	0.0001	0.0005
European	Mean-Feature Size	11.5667	12.4667	13.9667	12.2667	11	12.6667	14.4	15.3667	11.6	13.3333	13.5667	13.1	13
	SD-Feature Size	2.5908	1.5217	2.8105	2.6575	2.5298	3.1234	3.1896	2.4424	2.6658	2.8087	2.6543	2.4947	2.3238
Australian	Mean-Feature Size	6.2333	6.3333	7.4333	6.3667	6.0333	6.2667	7.4333	7.6333	6.4667	6.8	6.9667	7	6.9667
	SD-Feature Size	1.1455	0.8692	1.2828	0.836	0.9123	0.7272	1.8562	1.2243	0.8055	0.9092	1.2243	1.1832	1.378

Table 6 (continued)

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPSO
PaySim	Mean-Feature Size	2.9667	3	3.5333	2.9667	3	3	3.4667	3.6333	2.9	2.8333	3.2667	2.9	2.7667
European	SD-Feature Size	0.1795	0	0.9568	0.1795	0	0	0.8844	0.6574	0.3	0.3727	0.7272	0.3	0.5588
	Mean-MCC	0.8966	0.8909	0.8652	0.8963	0.8942	0.8918	0.8641	0.8706	0.8917	0.877	0.867	0.8854	0.8775
Australian	SD-MCC	0.0064	0.007	0.0124	0.009	0.0071	0.0065	0.0126	0.0055	0.0092	0.0067	0.0094	0.0062	0.0069
	Mean-MCC	0.7922	0.7902	0.7516	0.788	0.7896	0.787	0.7378	0.7547	0.7809	0.7697	0.7565	0.7861	0.7718
PaySim	SD-MCC	0.007	0.0088	0.0165	0.0098	0.0079	0.0096	0.0164	0.0133	0.013	0.0093	0.0193	0.0102	0.012
	Mean-MCC	0.9829	0.983	0.9805	0.9829	0.983	0.983	0.9815	0.9817	0.9827	0.9826	0.981	0.9827	0.9819
European	SD-MCC	0.0004	0	0.0036	0.0004	0	0	0.0012	0.0011	0.0008	0.0009	0.0021	0.0007	0.001
	Mean-CCI	878	879	877	879	878	878	877	877	878	877	877	878	878
Australian	SD-CCI	1.5567	1.1743	1.8163	1.5308	1.5649	1.578	1.667	1.2776	1.2512	1.0567	1.464	1.3888	1.4742
	Mean-CCI	598	598	601	598	598	598	601	600	599	599	600	599	599
PaySim	SD-CCI	1.6519	1.9619	2.6043	1.8607	1.82	1.7461	3.5884	2.1561	2.1071	1.82	2.6575	2.2171	1.655
	Mean-CCI	14,793	14,795	14,786	14,793	14,795	14,795	14,782	14,788	14,790	14,787	14,775	14,790	14,777
European	SD-CCI	8.2573	0	19.0398	8.2573	0	0	20.3026	15.4548	13.7535	17.1069	23.3034	13.7535	21.6481
	Mean-ICI	105	104	106	104	105	105	106	106	105	106	106	105	105
Australian	SD-ICI	1.5567	1.1743	1.8163	1.5308	1.5649	1.578	1.667	1.2776	1.2512	1.0567	1.464	1.3888	1.4742
	Mean-ICI	91	91	88	91	91	91	88	89	90	90	89	90	90
PaySim	SD-ICI	1.6519	1.9619	2.6043	1.8607	1.82	1.7461	3.5884	2.1561	2.1071	1.82	2.6575	2.2171	1.655
	Mean-ICI	1632	1631	1639	1632	1631	1631	1643	1637	1635	1638	1650	1635	1648
	SD-ICI	8.2573	0	19.0398	8.2573	0	0	20.3026	15.4548	13.7535	17.1069	23.3034	13.7535	21.6481

The bold values indicate the highest values achieved by the algorithms

Table 7 BGVOA-LS versus optimization algorithms for CCFD via the Xgb-tree classifier

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHGO	BSFO	BBA	BGOA	BPSSO
European	Mean-Accuracy	0.9601	0.9577	0.9459	0.9579	0.9579	0.9567	0.9489	0.9492	0.9548	0.9508	0.9479	0.9558	0.9525
	SD-Accuracy	0.0029	0.0033	0.0042	0.0037	0.0037	0.0034	0.0043	0.0035	0.004	0.0027	0.0051	0.003	0.0033
Australian	Mean-Accuracy	0.9135	0.907	0.8833	0.9077	0.9039	0.9053	0.8778	0.887	0.9022	0.8949	0.8853	0.9027	0.8969
	SD-Accuracy	0.0067	0.0065	0.0094	0.007	0.0049	0.0075	0.0091	0.0078	0.0074	0.0079	0.0075	0.0072	0.0081
PaySim	Mean-Accuracy	0.9985	0.9985	0.9983	0.9985	0.9985	0.9985	0.9981	0.9982	0.9985	0.9985	0.9981	0.9984	0.9984
	SD-Accuracy	0.0000	0.0000	0.0004	0.0000	0.0000	0.0000	0.0005	0.0005	0.0000	0.0000	0.0006	0.0002	0.0002
European	Mean-Precision	0.9722	0.9714	0.9627	0.9701	0.9707	0.9696	0.9737	0.9646	0.9696	0.9654	0.9641	0.9689	0.9668
	SD-Precision	0.0075	0.0082	0.0095	0.0088	0.0079	0.0064	0.0433	0.0096	0.0079	0.0081	0.008	0.0065	0.0088
Australian	Mean-Precision	0.9102	0.8853	0.8629	0.8934	0.8846	0.8849	0.7513	0.8659	0.8851	0.8769	0.8616	0.885	0.8755
	SD-Precision	0.0244	0.0215	0.0241	0.0272	0.0176	0.0236	0.1574	0.0211	0.0189	0.0246	0.0268	0.0226	0.0236
PaySim	Mean-Precision	1.0000	1.0000	0.9998	1.0000	1.0000	1.0000	1.4312	0.9998	1.0000	1.0000	0.9998	1.0000	1.0000
	SD-Precision	0.0000	0.0000	0.0003	0.0000	0.0000	0.0000	1.0036	0.0003	0.0000	0.0000	0.0003	0.0001	0.0002
European	Mean-Recall	0.9469	0.9429	0.9272	0.9446	0.9439	0.9425	0.9303	0.9323	0.9388	0.9347	0.9299	0.9415	0.9367
	SD-Recall	0.0072	0.009	0.0094	0.0086	0.0094	0.0089	0.0127	0.011	0.0099	0.0094	0.0082	0.0095	0.0103
Australian	Mean-Recall	0.851	0.8608	0.815	0.8536	0.8516	0.8562	0.8105	0.8222	0.8458	0.834	0.8235	0.8477	0.8418
	SD-Recall	0.0157	0.0192	0.0302	0.0207	0.0187	0.0211	0.0279	0.0208	0.0194	0.0219	0.03	0.0213	0.0263
PaySim	Mean-Recall	0.9969	0.9969	0.9967	0.9969	0.9969	0.9969	0.9965	0.9966	0.9969	0.9969	0.9964	0.9969	0.9969
	SD-Recall	0.0000	0.0000	0.0005	0.0000	0.0000	0.0000	0.0009	0.0007	0.0000	0.0000	0.0011	0.0002	0.0003
European	Mean-F1 Score	0.9593	0.9568	0.9446	0.9571	0.9571	0.9558	0.9477	0.9481	0.9539	0.9497	0.9467	0.955	0.9514
	SD-F1 Score	0.0029	0.0034	0.0043	0.0038	0.0039	0.0036	0.0047	0.0037	0.0042	0.0028	0.0052	0.0032	0.0035
Australian	Mean-F1 Score	0.8792	0.8725	0.8376	0.8725	0.8675	0.8699	0.8305	0.8431	0.8647	0.8544	0.8413	0.8655	0.8577
	SD-F1 Score	0.0081	0.0085	0.014	0.0083	0.0069	0.0099	0.0128	0.0108	0.0104	0.0101	0.0104	0.0096	0.0114
PaySim	Mean-F1 Score	0.9985	0.9985	0.9982	0.9985	0.9985	0.9985	0.9981	0.9982	0.9985	0.9985	0.9981	0.9984	0.9984
	SD-F1 Score	0.0000	0.0000	0.0004	0.0000	0.0000	0.0000	0.0006	0.0005	0.0000	0.0000	0.0006	0.0002	0.0002
European	Mean-SPC	0.9731	0.9724	0.9643	0.971	0.9717	0.9707	0.9673	0.966	0.9707	0.9667	0.9657	0.97	0.968
	SD-SPC	0.0075	0.0082	0.0097	0.0089	0.008	0.0066	0.0093	0.0099	0.008	0.0083	0.0081	0.0066	0.0091

Table 7 (continued)

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPSO
Australian	Mean-SPC	0.9502	0.9341	0.9234	0.9395	0.9345	0.9341	0.9172	0.9249	0.9352	0.9307	0.9215	0.9349	0.9291
PaySim	SD-SPC	0.0155	0.0145	0.0174	0.0185	0.0123	0.016	0.0179	0.0145	0.0124	0.0177	0.0199	0.0155	0.0168
	Mean-SPC	1.0000	1.0000	0.9998	1.0000	1.0000	1.0000	0.9998	0.9998	1.0000	1.0000	0.9998	1.0000	1.0000
European	SD-SPC	0.0000	0.0000	0.0003	0.0000	0.0000	0.0000	0.0003	0.0003	0.0000	0.0000	0.0003	0.0001	0.0002
	Mean-Kappa	0.9201	0.9154	0.8917	0.9157	0.9157	0.9134	0.8978	0.8985	0.9096	0.9015	0.8957	0.9117	0.9049
Australian	SD-Kappa	0.0057	0.0066	0.0084	0.0075	0.0075	0.0068	0.0087	0.0069	0.008	0.0053	0.0101	0.006	0.0067
	Mean-Kappa	0.812	0.7993	0.7467	0.8003	0.7921	0.7955	0.735	0.7549	0.7881	0.7723	0.7516	0.7893	0.7769
PaySim	SD-Kappa	0.0137	0.0136	0.0207	0.014	0.0105	0.0157	0.0195	0.0168	0.0161	0.0163	0.0158	0.0152	0.0174
	Mean-Kappa	0.997	0.997	0.9965	0.997	0.997	0.997	0.9963	0.9964	0.997	0.997	0.9962	0.9969	0.9968
European	SD-Kappa	0.0000	0.0000	0.0008	0.0000	0.0000	0.0000	0.0011	0.0010	0.0000	0.0000	0.0013	0.0003	0.0005
	Mean-ROC	0.9798	0.9794	0.9774	0.9789	0.9779	0.978	0.9757	0.9771	0.978	0.9765	0.9755	0.9795	0.9777
Australian	SD-ROC	0.004	0.0035	0.0043	0.0041	0.0047	0.004	0.0049	0.0049	0.0044	0.0047	0.0052	0.0042	0.0038
	Mean-ROC	0.9152	0.9142	0.9061	0.9131	0.9087	0.9122	0.901	0.9034	0.9099	0.9063	0.907	0.907	0.9071
PaySim	SD-ROC	0.0047	0.0066	0.0113	0.0047	0.0092	0.0085	0.0104	0.009	0.0089	0.0106	0.0116	0.0121	0.0108
	Mean-ROC	0.9988	0.9988	0.9989	0.9988	0.9988	0.9988	0.9989	0.9989	0.9988	0.9988	0.9988	0.9988	0.9988
European	SD-ROC	0.0001	0.0001	0.0002	0.0001	0.0001	0.0001	0.0003	0.0003	0.0001	0.0001	0.0002	0.0001	0.0001
	Mean-Fitness	0.0441	0.0464	0.0586	0.0466	0.0462	0.0473	0.0555	0.0558	0.0494	0.0537	0.0563	0.0484	0.0518
Australian	SD-Fitness	0.0028	0.0031	0.004	0.004	0.0036	0.0034	0.0039	0.0032	0.0039	0.002	0.0048	0.0027	0.0033
	Mean-Fitness	0.0903	0.0966	0.1206	0.0959	0.0996	0.0984	0.1264	0.1179	0.1016	0.1094	0.1183	0.1011	0.1066
PaySim	SD-Fitness	0.0062	0.006	0.0093	0.0068	0.005	0.0071	0.0095	0.0078	0.0072	0.0075	0.007	0.007	0.0079
	Mean-Fitness	0.0048	0.0048	0.0055	0.0048	0.0048	0.0048	0.0056	0.0056	0.0048	0.0048	0.0053	0.0049	0.0049
European	SD-Fitness	0.0000	0.0000	0.0006	0.0000	0.0000	0.0000	0.0006	0.0005	0.0000	0.0000	0.0005	0.0002	0.0002
	Mean-Feature Size	13.6	13.5667	15.0333	14.5333	13.3667	13.3333	14.6	16.6333	14	14.7333	14.2	13.9	14.3
Australian	SD-Feature Size	27154	1.961	2.8923	2.5526	2.5623	2.454	3.3126	2.3732	2.6957	2.4212	2.4953	2.797	2.1471
	Mean-Feature Size	6.5667	64	72	64	6.2333	6.4667	7.5333	8.3333	6.6	7.4667	6.6333	6.5667	6.2667
	SD-Feature Size	1.585	1.9596	1.9044	1.7626	1.9947	1.7651	2.2618	1.4907	1.9933	1.668	1.538	1.4985	1.672

Table 7 (continued)

Datasets	Metrics	BGVOA-LS	BGVOA	BMOA	BAVO	BAO	BSSA	BASO	BHGSO	BHHO	BSFO	BBA	BGOA	BPPO
PaySim	Mean-Feature Size	3.0000	3.0000	3.3667	3.0000	3.0000	3.0000	3.3333	3.4	3.0000	3.0000	3.1333	3.0000	3.0000
European	SD-Feature Size	0.0000	0.0000	0.4819	0.0000	0.0000	0.0000	0.6498	0.5538	0.0000	0.0000	0.6182	0.0000	0.0000
	Mean-MCC	0.9205	0.9159	0.8924	0.9161	0.9162	0.9138	0.8986	0.8991	0.9102	0.9021	0.8964	0.9121	0.9055
Australian	SD-MCC	0.0058	0.0065	0.0084	0.0075	0.0074	0.0067	0.0083	0.0069	0.0079	0.0053	0.0101	0.0058	0.0066
	Mean-MCC	0.8135	0.8	0.7482	0.8014	0.7928	0.7963	0.7363	0.7559	0.789	0.7735	0.753	0.7903	0.7779
PaySim	SD-MCC	0.0146	0.0139	0.0204	0.0146	0.0106	0.0161	0.0194	0.0171	0.0162	0.0165	0.0158	0.0155	0.0174
	Mean-MCC	0.997	0.997	0.9965	0.997	0.997	0.997	0.9963	0.9964	0.997	0.997	0.9963	0.9969	0.9968
European	SD-MCC	0.0000	0.0000	0.0008	0.0000	0.0000	0.0000	0.0011	0.0010	0.0000	0.0000	0.0013	0.0003	0.0005
	Mean-CCI	882	882	881	882	882	882	881	881	881	881	881	882	881
Australian	SD-CCI	1.335	1.5649	1.6852	1.5638	1.5524	1.3888	2.01	1.9482	1.578	1.6603	1.2579	1.4985	1.7978
	Mean-CCI	599	601	600	600	601	601	600	600	600	600	600	600	601
PaySim	SD-CCI	2.0155	2.0734	2.7651	2.5482	1.9102	2.2598	2.7047	2.0613	1.8015	2.4588	3.0993	2.2413	2.5736
	Mean-CCI	14,772	14,772	14,771	14,772	14,772	14,772	14,771	14,771	14,772	14,772	14,771	14,771	14,771
European	SD-CCI	0.0000	0.0000	0.6289	0.0000	0.0000	0.0000	1.0562	0.7951	0.0000	0.0000	1.3536	0.1795	0.2494
	Mean-ICI	101	101	102	101	101	101	102	102	102	102	102	101	102
Australian	SD-ICI	1.335	1.5649	1.6852	1.5638	1.5524	1.3888	2.01	1.9482	1.578	1.6603	1.2579	1.4985	1.7978
	Mean-ICI	90	88	89	89	88	88	89	89	89	89	89	89	88
PaySim	SD-ICI	2.0155	2.0734	2.7651	2.5482	1.9102	2.2598	2.7047	2.0613	1.8015	2.4588	3.0993	2.2413	2.5736
	Mean-ICI	1654	1654	1654	1654	1654	1654	1654	1654	1654	1654	1654	1654	1654
PaySim	SD-ICI	0.0000	0.0000	0.6289	0.0000	0.0000	0.0000	1.0562	0.7951	0.0000	0.0000	1.3536	0.1795	0.2494

The bold values indicate the highest values achieved by the algorithms

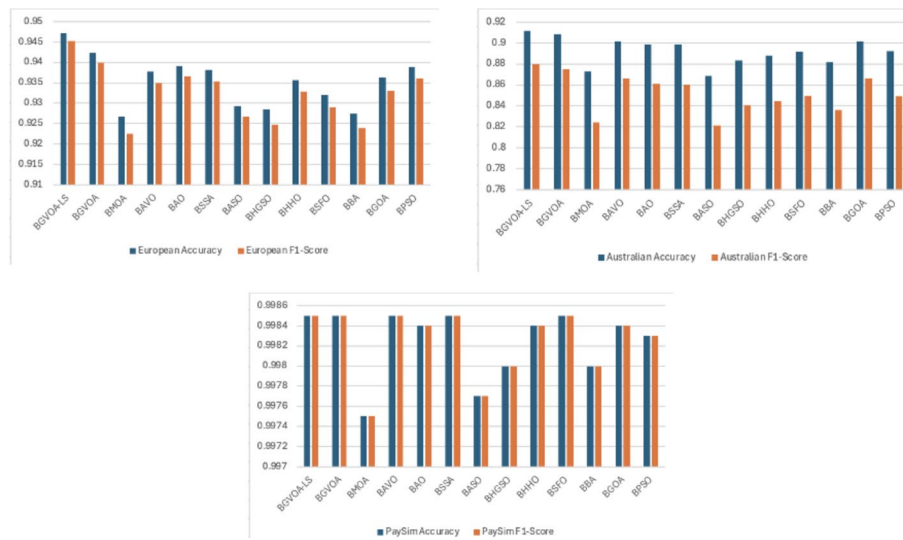


Fig. 3 BGVOA-LS versus optimization algorithms via the DT classifier

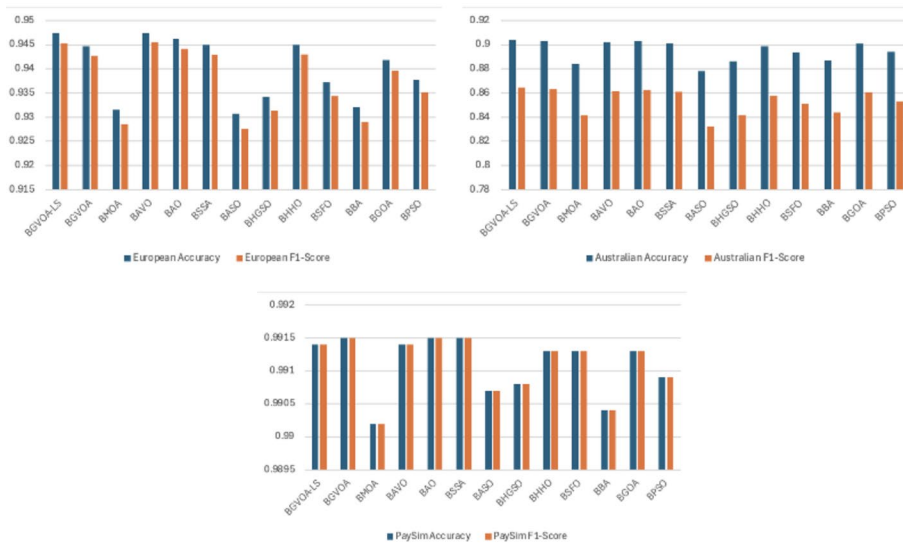


Fig. 4 BGVOA-LS versus optimization algorithms via the KNN classifier

algorithms by computing the mean accuracy and the F1 score for each dataset via the ML classifiers DT, KNN, and Xgb-tree, respectively.

The effectiveness of the BGVOA-LS algorithm is verified through comparative analysis against algorithms reported in recent studies, as detailed in Tables 8 and 9. This comparison evaluates critical performance metrics, including accuracy, F1 score, precision, and recall. The experimental results indicate that the BGVOA-LS algorithm is a robust and reliable solution for CCFD, achieving an effective balance between accuracy, precision, and recall that many advanced deep learning methods struggle to maintain. While autoencoders and similar approaches often report very high accuracy in controlled tests, their usefulness is limited by low precision and inconsistent recall,

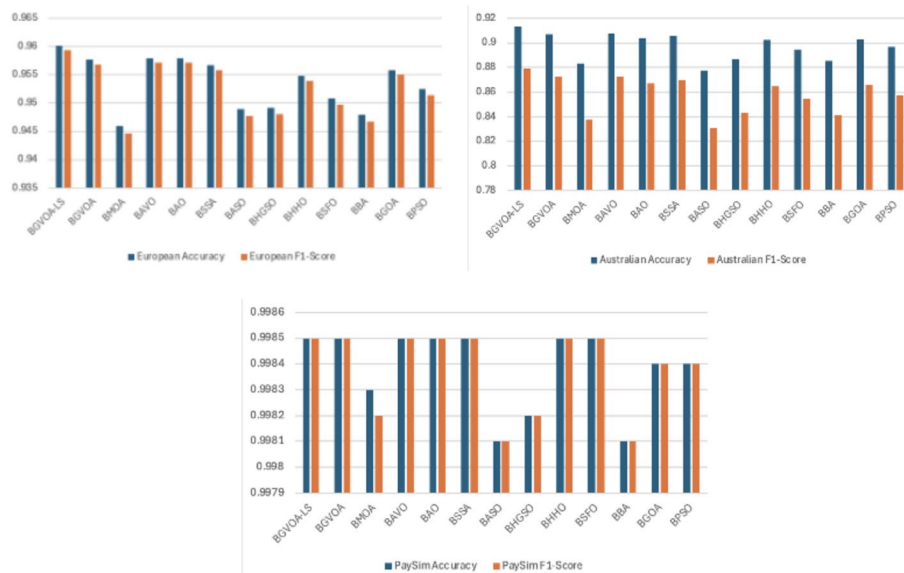


Fig. 5 BGVOA-LS versus optimization algorithms via the Xgb-tree classifier

Table 8 BGVOA-LS versus recent studies for CCFD using European datasets

Metrics	BGVOA-LS	JNBO-SpinalNet [36]	AFD-HHM [74]	GA Algorithm for FS [75]	PSLPSO [33]	HybridIG-CSO [76]	Autoencoder [30]	Autoencoder [22]
Accuracy	0.9472	0.8910	0.9400	0.9691	0.9100	0.9930	0.9974	0.9514
Precision	0.9783	0.8982	0.9400	0.7107	0.8700	0.995	0.2750	0.0309
Recall	0.9146	0.1020	0.9700	0.7610	0.9700	0.9890	0.2895	0.8367

Table 9 BGVOA-LS versus recent studies for CCFD using Australian datasets

Metrics	BGVOA-LS	BBBO [37]	GA-SMOTE-DCNN [77]	Stacked Classifier [78]	HybridIG-CSO [76]	XGBoost + GAFS [79]	FFSVM [80]
Accuracy	0.9118	0.8879	0.8823	0.8623	0.837	0.8993	0.8565
Precision	0.8848	0.8151	0.863	N/A	0.609	N/A	0.795
Recall	0.8758	0.9013	0.879	N/A	0.672	N/A	0.912

which results in excessive false positives and reduced reliability in real-world settings. In contrast, BGVOA-LS consistently delivers strong accuracy together with high precision, reducing costly false alarms and solid recall, ensuring that fraudulent activities are detected across different datasets. This consistent performance highlights its practical strength in establishing BGVOA-LS not only as a research contribution but also as a dependable, real-world solution that combines competitive results with operational efficiency, making it more suitable for deployment than accuracy-focused deep learning models.

A detailed analysis of the proposed BGVOA-LS algorithm is presented in this section, aimed at evaluating its convergence capabilities in addressing the CCFD problem

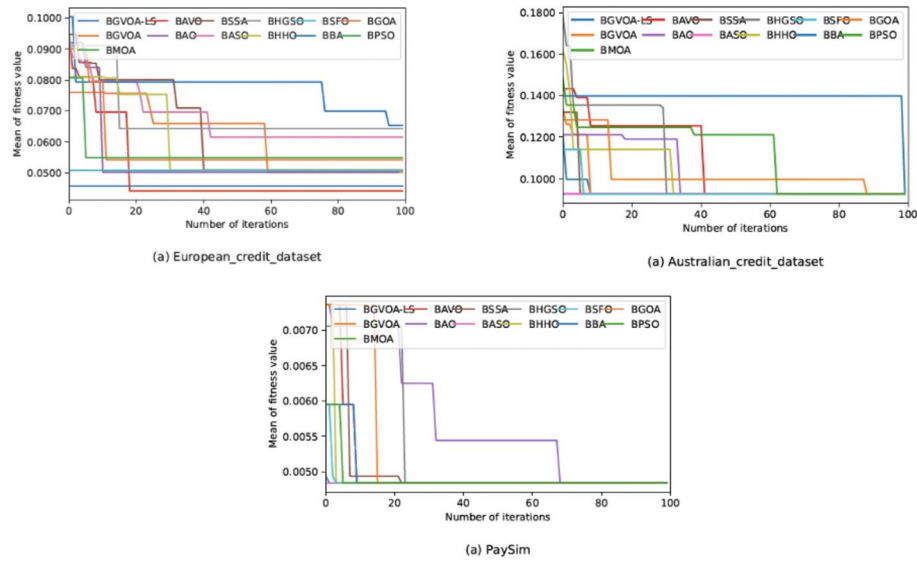


Fig. 6 BGVOA-LS convergence graphs versus MHTs in the DT classifier

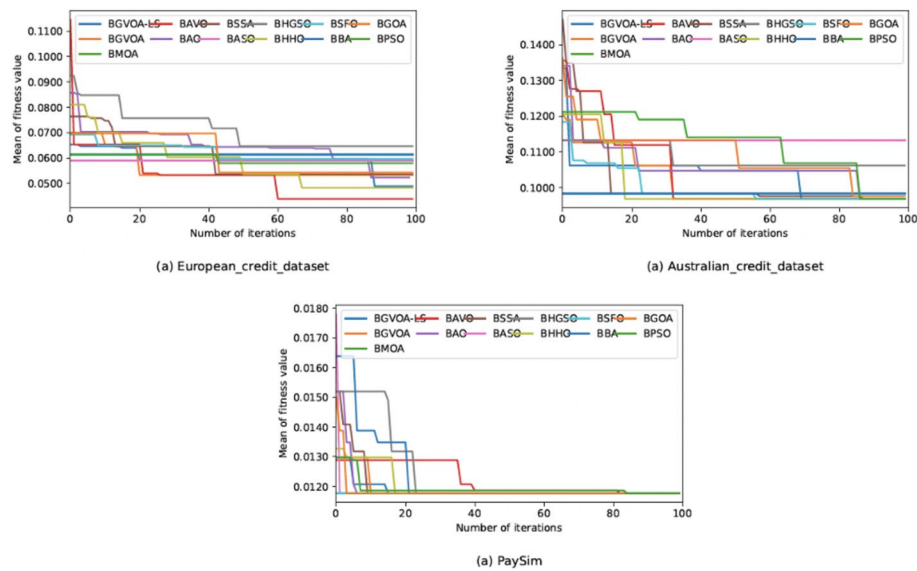


Fig. 7 BGVOA-LS convergence graphs versus MHTs in the KNN classifier

via the selected dataset. The performance of the BGVOA-LS algorithm was assessed via different ML classifiers: DT, KNN, and Xgb-tree. Figures 6, 7, and 8 illustrate the convergence behavior of the BGVOA-LS algorithm in combination with each classifier. These convergences provide clear insights into the efficiency and effectiveness of the proposed BGVOA-LS algorithm by comparing its performance against competitive algorithms under identical experimental conditions, including the same number of iterations and population sizes; the figures demonstrate that the BGVOA-LS algorithm achieves rapid convergence while maintaining optimal accuracy on the chosen dataset. The BGVOA-LS algorithm exhibits strong proficiency in rapidly finding optimal solutions, effectively balancing its exploration and exploitation processes, and

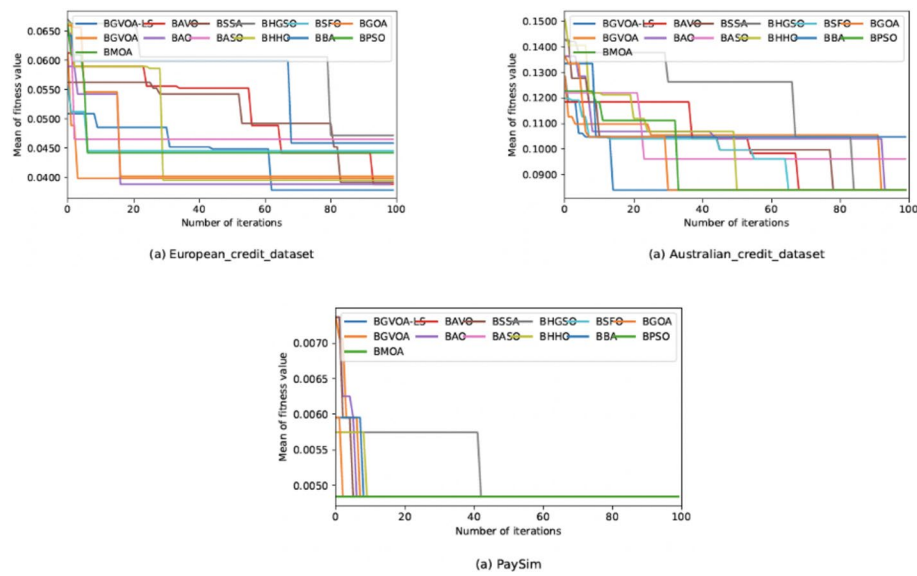


Fig. 8 BGVOA-LS convergence graphs versus MHTs in the Xgb-tree classifier

Table 10 Wilcoxon test: BGVOA-LS performance metrics with DT

BGVOA-LS with DT versus	R +	R-	P_Value	Winner
BGVOA	100.0	5.0	0.0012208	BGVOA-LS
BMOA	120.0	0.0	6.104E-5	BGVOA-LS
BAVO	100.0	5.0	0.0012208	BGVOA-LS
BAO	105.0	0.0	1.2208E-4	BGVOA-LS
BSSA	100.0	5.0	0.0012208	BGVOA-LS
BASO	120.0	0.0	6.104E-5	BGVOA-LS
BHGSO	120.0	0.0	6.104E-5	BGVOA-LS
BHHO	105.0	0.0	1.2208E-4	BGVOA-LS
BSFO	100.0	5.0	0.0012208	BGVOA-LS
BBA	120.0	0.0	6.104E-5	BGVOA-LS
BGOA	103.5	1.5	3.05205E-4	BGVOA-LS
BPSO	114.0	6.0	8.544E-4	BGVOA-LS

validating its practical applicability and robustness in handling complex optimization scenarios within fraud detection frameworks.

A comprehensive comparative analysis was conducted via the Wilcoxon test to validate the effectiveness of the proposed BGVOA-LS algorithm by conducting the Wilcoxon-sum test, a robust nonparametric statistical method [81]. The test assesses significant performance differences by ranking absolute differences between algorithm performances and calculating rank sums. Tables 10, 11, and 12 consistently indicate the superior performance of BGVOA-LS with statistically significant outcomes ($p_values < 0.05$), underscoring its effectiveness in handling complex classification tasks and its potential for practical application by comparing the BGVOA-LS with DT, KNN, and Xgb-Tree, respectively.

Table 11 Wilcoxon test: BGVOA-LS performance metrics with KNN

BGVOA-LS with KNN versus	R+	R-	P_Value	Winner
BGVOA	111.0	9.0	0.002014	BGVOA-LS
BMOA	115.0	5.0	6.104E-4	BGVOA-LS
BAVO	92.0	28.0	0.073	BGVOA-LS
BAO	96.0	9.0	0.004028	BGVOA-LS
BSSA	112.0	8.0	0.0015258	BGVOA-LS
BASO	104.0	16.0	0.010254	BGVOA-LS
BHGSO	113.0	7.0	0.0011596	BGVOA-LS
BHHO	111.0	9.0	0.002014	BGVOA-LS
BSFO	113.0	7.0	0.0011596	BGVOA-LS
BBA	111.0	9.0	0.002014	BGVOA-LS
BGOA	113.0	7.0	0.0011596	BGVOA-LS
BPSO	111.0	9.0	0.002014	BGVOA-LS

Table 12 Wilcoxon test: BGVOA-LS performance metrics with Xgb-tree

BGVOA-LS with Xgb-tree versus	R+	R-	P_Value	Winner
BGVOA	88.0	17.0	0.02454	BGVOA-LS
BMOA	120.0	0.0	6.104E-5	BGVOA-LS
BAVO	90.0	15.0	0.016602	BGVOA-LS
BAO	95.0	10.0	0.00525	BGVOA-LS
BSSA	90.0	15.0	0.016602	BGVOA-LS
BASO	100.0	20.0	0.02154	BGVOA-LS
BHGSO	120.0	0.0	6.104E-5	BGVOA-LS
BHHO	100.0	5.0	0.0012208	BGVOA-LS
BSFO	100.0	5.0	0.0012208	BGVOA-LS
BBA	120.0	0.0	6.104E-5	BGVOA-LS
BGOA	103.5	1.5	3.0525E-4	BGVOA-LS
BPSO	103.0	1.5	3.0525E-4	BGVOA-LS

Real-time integration feasibility

FD systems operate under strict timing requirements that often require classification within a few milliseconds. The feature reduction provided by BGVOA-LS directly helps in this setting, since smaller input dimensions shorten the inference time and reduce the chance of bottlenecks during transaction scoring. This makes the framework a practical candidate for real-time or near-real-time monitoring pipelines.

The reduction in feature space also lowers memory use, an essential factor when models are deployed in production. Leaner representations allow efficient handling of high-volume data streams and enable edge deployment, where memory and computational resources are limited, such as at POS or gateway devices.

In terms of computational resources, BGVOA-LS is flexible. For moderate data loads, CPUs can deliver acceptable performance. At the scale of large financial networks, GPU acceleration or distributed servers become necessary to sustain high throughput while

keeping latency low; this scalability ensures that the method can adapt to different infrastructure settings. Finally, BGVOA-LS can be applied in both batch and streaming contexts. Batch processing is well-suited for periodic retraining and FS, whereas streaming inference supports continuous decision-making during live transactions, where batch updates feed into a streaming detection pipeline and can balance adaptability with responsiveness.

Although this study does not include direct simulation of streaming data, the efficiency gains observed in the experimental results indicate that BGVOA-LS is structurally well aligned with the requirements of real-time fraud detection. Future empirical validation of live transaction streams would be a natural extension of this work.

Discussions

The experimental results demonstrate the performance of BGVOA-LS over traditional MHTs and ML classifiers revealing broader insights into its practical impact and theoretical contributions through a deep examination. The consistent performance of the proposed algorithm across all three datasets (European, Australian, and PaySim) highlights its robustness and generalizability across varying financial environments, indicating its suitability for real-world deployment where fraud patterns differ significantly.

Additionally, the substantial improvement in accuracy and reduction in selected features not only validate the effectiveness of the hybrid optimization and FS mechanism but also underscore the ability of the proposed algorithms to maintain high performance with reduced model complexity. This balance is critical in operational settings where computational efficiency directly translates to cost savings and faster decision making.

The superior MCC and AUC-ROC scores suggest that BGVOA-LS is particularly effective in handling class imbalance, a notorious challenge in CCFD. This indicates that the model not only performs well under balanced conditions but also excels where most fraud detection systems fail to detect rare but critical fraudulent cases.

The local search refinement and binary transformation strategy embedded in BGVOA-LS must reflect an evolutionary leap from heuristic-based optimization to biologically inspired, intelligent feature engineering. This innovation opens avenues for future research on integrating specific domain knowledge with adaptive learning to create even more aware fraud detection systems. By examining the results beyond just the numerical tables, we confirm the strong performance of the proposed algorithm and add to the broader understanding of what makes fraud detection systems both effective and scalable. This includes key factors such as adaptability to different datasets, efficient use of computational resources, and strong performance even when dealing with imbalanced data.

Conclusion and future work

This study provides strong evidence for the effectiveness of the BGVOA-LS model in advancing computational credit fraud detection by embedding an LS with an optimization algorithm; the model delivers not only predictive excellence but also operational

efficiency. This dual advantage is especially valuable for financial institutions that strive to meet the increasing demands for both speed and accuracy in fraud monitoring.

From a managerial standpoint, implementing BGVOA-LS can transform fraud risk management strategies because of its ability to drastically reduce the dimensionality of input data while preserving accuracy, resulting in faster decision-making, lower infrastructure costs, and increased compliance readiness. In addition, the proposed algorithm of transparency enables internal auditors and regulators to understand the rationale behind flagged transactions, which aligns well with broader corporate governance goals of accountability and risk mitigation. The proposed model supports scalable deployment in diverse environments, from traditional banking systems to digital wallets and mobile transactions. BGVOA-LS not only contributes to the academic literature on CCFD but also provides actionable tools for institutions aiming to proactively combat evolving fraud schemes with intelligence-driven solutions.

The proposed BGVOA-LS algorithm has demonstrated strong performance across benchmark datasets indicating several promising future directions that can further extend its impact in real-world fraud detection systems. One key area is real-time deployment, where systematically evaluating inference latency, memory consumption and resource utilization in streaming environments such as Apache Kafka or Spark is essential. Future work will also explore the feasibility of deployment on edge devices and GPU-accelerated platforms, ensuring scalability across both lightweight and enterprise-scale fraud detection pipelines by enhancing its capabilities through integration with DL models (e.g., CNN-LSTM, transformer-based encoders), which improve the detection of complex fraud behaviors. A multiobjective version of the BGVOA-LS may be optimized for various practical concerns beyond accuracy and feature reduction, such as latency, interpretability, and energy efficiency. Adapting the algorithm for federated learning would support privacy-preserving collaboration across institutions without compromising sensitive data, as would mechanisms for robustness against malicious attacks and concept drift, which would be incorporated to handle evolving fraud tactics; this would ensure long-term adaptability. Finally, combining BGVOA-LS with explainable artificial intelligence (AI) tools, such as SHAP or LIME, can increase transparency, foster trust, and ensure adherence to financial regulations, including the GDPR and PSD2.

Appendix A

This appendix provides detailed configuration settings for the metaheuristic techniques and the machine learning classifiers employed in this study. These supplementary tables support the comparative analysis outlined in the main text (Tables 13 and 14).

Table 13 MHT configuration settings and coefficients

MHTs Algorithms	Parameters
All Algorithms	Runs No. = 30 Iterations No. MaxIt = 100 Population size N = 30 Dimensionality d = The features number in the used dataset
BGVOA	ω = Decreased linearly from 1 to 0 C1 = Either a standard normal or uniform random number b = Randomly changes within (0, 4) a = Decreased quadratically from 1 to 0
BMOA	$senry = 0.3$ $P = 0.5$ $\beta = 1.5$ $r = 1$
BAVO	Parameter $L_1 = 0.7$ Parameter $L_2 = 0.2$ Parameter $w = 2$ Parameter $P_1 = 0.6$ Parameter $P_2 = 0.6$ Parameter $P_3 = 0.5$
BAO	Number of search rotations $r_1 = 10$ $U = 0.00565$ $\omega = 0.005$ Adjustment parameters for exploitation stage $\alpha = 0.1$ and $\delta = 0.1$ Aquila's arbitrary motions $Q_1 \in [-1, 1]$ Aquila's flying slope $Q_2 \in [2, 0]$
BSSA	Number of scroungers $SD = 0.1 * N$ Number of producers $PD = 0.2 * N$ Safety threshold $ST = 0.8$
BASO	Multiplier weight $\beta = 0.2$ Depth weight $\alpha = 50$
BHGSO	Number of clusters = 2 $L_1 = 5E - 03$, $L_2 = 1E + 02$, and $L_3 = 1E - 02$ $\beta = \alpha = 0.1$ and $K = 1$
BHHO	Rabbit energy $E \in [-1, 1]$
BSFO	Ratio between sardines and sailfish $pp = 0.1$ $A = 1$ $\varepsilon = 0.0001$
BBA	Loudness $A = 0.8$ Pulse emission rate $r = 0.95$ Lower and upper pulse frequencies = 0, 10
BGOA	$C_{min} = 0.00004$ and $C_{max} = 1$
BPSO	Acceleration coefficients ($C_1 = C_2 = 1.2$) Inertia weight $\omega_{max} = 0.9$, $\omega_{min} = 0.4$

Table 14 ML classifier’s configuration settings and coefficients

ML Classifier	Parameters
KNN	Euclidean distance $k=5$
DT	Maximum depth $max_depth=5$ No. of attributes $max_features=1$
Xgb-Tree	Maximum depth $max_depth=3$ Minimum loss reduction $gamma=0$ Minimum sum of instance weight $min_child_weight=1$ No. of boosting rounds $nrounds=100$ Learning rate $eta=0.4$ Subsample ratio of learning $sub_sample=0.75$ Subsample ratio of columns $colsample_bytree=0.8$

Acknowledgements

The authors gratefully acknowledge Damanhour University, Egypt, Faculty of Computer and Information, for providing access to research laboratories and facilities, which enabled the experimental work for this manuscript

Author contributions

AHS: Conceptualization, methodology, software, formal analysis, visualization, writing original draft preparation, and designing all figures and tables SMA: Supervised the manuscript content OEE: Supervised the manuscript content AAA: Supervision, Validation, Visualization, Review and Editing the manuscript content All authors read and approved the final manuscript. Authors: Aya Hesham Salem, Safaa Magdy Azzam, Osama Emam from Helwan University Amr Abohany from Faculty of Computer and Information, Damanhour University, Damanhour, Egypt.

Funding

Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB).

Data availability

Data will be available upon request.

Declarations

Competing interests

The authors declare no competing interests.

Received: 19 May 2025 Accepted: 7 September 2025

Published online: 17 September 2025

References

1. https://www.ftc.gov/news-events/news/press-releases/2025/03/new-ftc-data-show-big-jump-reported-losses-fraud-125-billion-2024?utm_source=chatgpt.com.
2. <https://www.statista.com/outlook/fmo/payments/digital-payments/worldwide?currency=USD>.
3. Salem AH, Azzam SM, Emam OE, Abohany AA. Advancing cybersecurity: a comprehensive review of AI-driven detection techniques. *J Big Data*. 2024. <https://doi.org/10.1186/s40537-024-00957-y>.
4. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.
5. <https://www.kaggle.com/datasets/mtalaltariq/paysim-data>.
6. <https://archive.ics.uci.edu/dataset/143/statlog+australian+credit+approval>.
7. Lee SW, et al. A survey of Beluga whale optimization and its variants: statistical analysis, advances, and structural reviewing. *Comput Sci Rev*. 2025. <https://doi.org/10.1016/j.cosrev.2025.100740>.
8. Gharehchopogh FS. An improved boosting bald eagle search algorithm with improved African vultures optimization algorithm for data clustering. *Ann Data Sci*. 2025;12(2):605–37. <https://doi.org/10.1007/s40745-024-00525-4>.
9. Aribowo W, et al. Enhancing photovoltaic parameters based on modified puma optimizer. *Bull Electr Eng Inform*. 2025;14(3):1667–80. <https://doi.org/10.11591/eei.v14i3.8977>.
10. Lee SW, et al. A hybrid flow shop scheduling optimization using puma optimizer: addressing energy efficiency and workforce restrictions. *Alexandria Eng J*. 2025;128:340–65. <https://doi.org/10.1016/j.aej.2025.05.071>.
11. El-Kenawy ESM, et al. Feature selection in wind speed forecasting systems based on meta-heuristic optimization. *PLoS ONE*. 2023. <https://doi.org/10.1371/journal.pone.0278491>.

12. Mohapatra S, Kaliyaperumal D, Gharehchopogh FS. A revamped black winged kite algorithm with advanced strategies for engineering optimization. *Sci Rep*. 2025. <https://doi.org/10.1038/s41598-025-93370-1>.
13. Anka F, Agaoglu N, Nematzadeh S, Torkamanian-afshar M, Gharehchopogh FS. Advances in artificial rabbits optimization: a comprehensive review. *Arch Comput Methods Eng*. 2025. <https://doi.org/10.1007/s11831-024-10202-7>.
14. Machado MR, Chen DT, Osterrieder JR. An analytical approach to credit risk assessment using machine learning models. *Decis Anal J*. 2025. <https://doi.org/10.1016/j.dajour.2025.100605>.
15. Hamada MA, Farag K, Abiche AE. Early prediction detection of retail and corporate credit risks using machine learning algorithms. *Emerg Sci J*. 2025;9(2):995–1012. <https://doi.org/10.28991/ESJ-2025-09-02-025>.
16. Darwish JA. Optimization and prediction of corporate credit rating through advanced feature selection based on AI and deep learning. *Alexandria Eng J*. 2025;127:586–94. <https://doi.org/10.1016/j.aej.2025.05.043>.
17. El-Kenawy ESM, et al. Metaheuristic optimization for improving weed detection in wheat images captured by drones. *Mathematics*. 2022. <https://doi.org/10.3390/math10234421>.
18. Hasan DO, Mohammed HM, Abdul ZK. Griffon vultures optimization algorithm for solving optimization problems. *Expert Syst Appl*. 2025;276:127206. <https://doi.org/10.1016/j.eswa.2025.127206>.
19. Alkanhel R, et al. Network intrusion detection based on feature selection and hybrid metaheuristic optimization. *Comput Mater Contin*. 2023;74(2):2677–93. <https://doi.org/10.32604/cmc.2023.033273>.
20. El-Kenawy ESM, Ibrahim A, Mirjalili S, Zhang YD, Elnazer S, Zaki RM. Optimized ensemble algorithm for predicting metamaterial antenna parameters. *Comput Mater Contin*. 2022;71(2):4989–5003. <https://doi.org/10.32604/cmc.2022.023884>.
21. Khodadadi N, Abualigah L, El-Kenawy ESM, Snasel V, Mirjalili S. An archive-based multi-objective arithmetic optimization algorithm for solving industrial engineering problems. *IEEE Access*. 2022;10:106673–98. <https://doi.org/10.1109/ACCESS.2022.3212081>.
22. Tonui I, Kibet P. Credit card fraud detection with deep learning techniques. [Online]. Available: <https://www.researchgate.net/publication/388459328>.
23. Yi Z, et al. Fraud detection in capital markets: a novel machine learning approach. *Expert Syst Appl*. 2023. <https://doi.org/10.1016/j.eswa.2023.120760>.
24. Ranganatha HR, Mustafa AS. Enhancing fraud detection efficiency in mobile transactions through the integration of bidirectional 3d Quasi-Recurrent Neural network and blockchain technologies. *Expert Syst Appl*. 2025. <https://doi.org/10.1016/j.eswa.2024.125179>.
25. Abdelghafour EB, Mohamed C, Noura A, Abdelhamid B. Enhancing credit card fraud detection using a stacking model approach and hyperparameter optimization. 2024. [Online]. Available: www.ijacsa.thesai.org.
26. Cherif A, Ammar H, Kalkatawi M, Alshehri S, Imine A. Encoder–decoder graph neural network for credit card fraud detection. *J King Saud Univ Comput Inform Sci*. 2024. <https://doi.org/10.1016/j.jksuci.2024.102003>.
27. Damanik N, Liu CM. Advanced fraud detection: leveraging K-SMOTEENN and stacking ensemble to tackle data imbalance and extract insights. *IEEE Access*. 2025. <https://doi.org/10.1109/ACCESS.2025.3528079>.
28. Alarfaj FK, Shahzadi S. Enhancing fraud detection in banking with deep learning: graph neural networks and autoencoders for real-time credit card fraud prevention. *IEEE Access*. 2024. <https://doi.org/10.1109/ACCESS.2024.3466288>.
29. Talukder MA, Khalid M, Uddin MA. An integrated multistage ensemble machine learning model for fraudulent transaction detection. *J Big Data*. 2024. <https://doi.org/10.1186/s40537-024-00996-5>.
30. Iqbal A, Amin R. Time series forecasting and anomaly detection using deep learning. *Comput Chem Eng*. 2024. <https://doi.org/10.1016/j.compchemeng.2023.108560>.
31. Karthikeyan T, Govindarajan M, Vijayakumar V. An effective fraud detection using competitive swarm optimization based deep neural network. *Meas Sensors*. 2023. <https://doi.org/10.1016/j.measen.2023.100793>.
32. Prabhakaran N, Nedunchelian R. Oppositional cat swarm optimization-based feature selection approach for credit card fraud detection. *Comput Intell Neurosci*. 2023. <https://doi.org/10.1155/2023/2693022>.
33. Mniai A, Tarik M, Jebbari K. Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000. A Novel Framework for Credit Card Fraud Detection <https://doi.org/10.1109/ACCESS.2017.DOI>.
34. Tekkali CG, Natarajan K. Smart fraud detection in E-transactions using synthetic minority oversampling and binary Harris hawks optimization. *Comput Mater Contin*. 2023;75(2):3171–87. <https://doi.org/10.32604/cmc.2023.036865>.
35. Tekkali CG, Natarajan K. Assessing CNN's performance with multiple optimization functions for credit card fraud detection. *Procedia Comput Sci*. 2024. <https://doi.org/10.1016/j.procs.2024.04.193>.
36. Reddy VVK, Reddy RVK, Munaga MSK, Karnam B, Maddila SK, Kolli CS. Deep learning-based credit card fraud detection in federated learning. *Expert Syst Appl*. 2024. <https://doi.org/10.1016/j.eswa.2024.124493>.
37. Sorour SE, AlBarak KM, Abohany AA, El-Mageed AAA. Credit card fraud detection using the brown bear optimization algorithm. *Alexandria Eng J*. 2024;104:171–92. <https://doi.org/10.1016/j.aej.2024.06.040>.
38. Krishnavardhan N, Govindarajan M, Achutha Rao SV. International Journal of INTELLIGENT SYSTEMS AND APPLICATIONS IN ENGINEERING Detection of Financial Fraudulent using Metaheuristic Algorithm with Support Vector Machine Based Classification. [Online]. Available: www.ijisae.org.
39. Niyazova K, Mukasheva A, Balbayev G, Iliev T, Mirambayeva N, Uzakbayev M. Ant colony optimization algorithm for feature selection in suspicious transaction detection system. *Eng Proc*. 2024. <https://doi.org/10.3390/engproc2024060018>.
40. Xia P, Zhu X, Charles V, Zhao X, Peng M. A novel heuristic-based selective ensemble prediction method for digital financial fraud risk. *IEEE Trans Eng Manag*. 2024;71:8002–18. <https://doi.org/10.1109/TEM.2024.3385298>.
41. Oak R. Detecting review fraud using metaheuristic graph neural networks. *Int J Inf Technol*. 2024. <https://doi.org/10.1007/s41870-024-01896-w>.
42. Vairetti C, Assadi JL, Maldonado S. Efficient hybrid oversampling and intelligent undersampling for imbalanced big data classification. *Expert Syst Appl*. 2024. <https://doi.org/10.1016/j.eswa.2024.123149>.
43. Elsoud EA et al. Under sampling techniques for handling unbalanced data with various imbalance rates: a comparative study. 2024. [Online]. Available: www.ijacsa.thesai.org.

44. Hancock JT, Wang H, Khoshgoftaar TM, Liang Q. Data reduction techniques for highly imbalanced Medicare big data. *J Big Data*. 2024. <https://doi.org/10.1186/s40537-023-00869-3>.
45. Harel R, Spiegel O, Getz WM, Nathan R. Social foraging and individual consistency in following behaviour: testing the information centre hypothesis in free-ranging vultures. *Proc R Soc Lond B Biol Sci*. 2017. <https://doi.org/10.1098/rspb.2016.2654>.
46. Spiegel O, Getz WM, Nathan R. Factors influencing foraging search efficiency: Why do scarce lappet-faced vultures outperform ubiquitous white-backed vultures? *Am Nat*. 2013. <https://doi.org/10.1086/670009>.
47. Spiegel O, Harel R, Getz WM, Nathan R. Mixed strategies of griffon vultures' (*Gyps fulvus*) response to food deprivation lead to a hump-shaped movement pattern. *Mov Ecol*. 2013. <https://doi.org/10.1186/2051-3933-1-5>.
48. Pirastru M, Mereu P, Manca L, Bebbere D, Naitana S, Leoni GG. Anthropogenic drivers leading to population decline and genetic preservation of the Eurasian griffon vulture (*Gyps fulvus*). *Life*. 2021. <https://doi.org/10.3390/life11101038>.
49. Tubishat M, Idris N, Shuib L, Abushariah MAM, Mirjalili S. Improved salp swarm algorithm based on opposition based learning and novel local search algorithm for feature selection. *Expert Syst Appl*. 2020. <https://doi.org/10.1016/j.eswa.2019.113122>.
50. Halder RK, Uddin MN, Uddin MA, Aryal S, Khraisat A. Enhancing k-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications. *J Big Data*. 2024. <https://doi.org/10.1186/s40537-024-00973-y>.
51. Chen T, Guestrin C. XGBoost: a scalable tree boosting system. 2016. <https://doi.org/10.1145/2939672.2939785>.
52. Rokach L, Maimon O. Data mining with decision trees. [Online]. Available: <http://www.worldscientific.com/series/smpai>.
53. Xian S, Feng X. Meerkat optimization algorithm: a new meta-heuristic optimization algorithm for solving constrained engineering problems. *Expert Syst Appl*. 2023. <https://doi.org/10.1016/j.eswa.2023.120482>.
54. Abdollahzadeh B, Gharehchopogh FS, Mirjalili S. African vultures optimization algorithm: a new nature-inspired metaheuristic algorithm for global optimization problems. *Computers Ind Eng*. 2021. <https://doi.org/10.1016/j.cie.2021.107408>.
55. Abualigah L, Yousri D, Elaziz MA, Ewees AA, Al-qaness MAA, Gandomi AH. Aquila optimizer: a novel meta-heuristic optimization algorithm. *Comput Ind Eng*. 2021. <https://doi.org/10.1016/j.cie.2021.107250>.
56. Xue J, Shen B. A novel swarm intelligence optimization approach: sparrow search algorithm. *Syst Sci Control Eng*. 2020;8(1):22–34. <https://doi.org/10.1080/21642583.2019.1708830>.
57. Zhao W, Wang L, Zhang Z. Atom search optimization and its application to solve a hydrogeologic parameter estimation problem. *Knowl Based Syst*. 2019;163:283–304. <https://doi.org/10.1016/j.knsys.2018.08.030>.
58. Hashim FA, Houssein EH, Mabrouk MS, Al-Atabany W, Mirjalili S. Henry gas solubility optimization: a novel physics-based algorithm. *Future Gener Comput Syst*. 2019;101:646–67. <https://doi.org/10.1016/j.future.2019.07.015>.
59. Heidari AA, Mirjalili S, Faris H, Aljarah I, Mafarja M, Chen H. Harris hawks optimization: algorithm and applications. *Future Gener Comput Syst*. 2019;97:849–72. <https://doi.org/10.1016/j.future.2019.02.028>.
60. Shadravan S, Naji HR, Bardsiri VK. The sailfish optimizer: a novel nature-inspired metaheuristic algorithm for solving constrained engineering optimization problems. *Eng Appl Artif Intell*. 2019;80:20–34. <https://doi.org/10.1016/j.engappai.2019.01.001>.
61. Mirjalili S, Mirjalili SM, Yang XS. Binary bat algorithm. *Neural Comput Appl*. 2014;25(3–4):663–81. <https://doi.org/10.1007/s00521-013-1525-5>.
62. Mirjalili SZ, Mirjalili S, Saremi S, Faris H, Aljarah I. Grasshopper optimization algorithm for multi-objective optimization problems. *Appl Intell*. 2018;48(4):805–20. <https://doi.org/10.1007/s10489-017-1019-8>.
63. Poli R, Kennedy J, Blackwell T. Particle swarm optimization. *Swarm Intell*. 2007;1(1):33–57. <https://doi.org/10.1007/s11721-007-0002-0>.
64. Torgo L, Ribeiro R. Precision and recall for regression. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. 2009. p. 332–346. https://doi.org/10.1007/978-3-642-04747-3_26.
65. Powers DM. Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation. *Artic J Mach Learn Technol*. 2011;2(1):37–63. <https://doi.org/10.9735/2229-3981>.
66. Hand DJ, Christen P, Kirielle N. F*: an interpretable transformation of the F-measure. *Mach Learn*. 2021;110(3):451–6. <https://doi.org/10.1007/s10994-021-05964-1>.
67. Silva FR, Vidotti VG, Cremasco F, Dias M, Gomi ES, Costa VP. Sensitivity and specificity of machine learning classifiers for glaucoma diagnosis using spectral domain OCT and standard automated perimetry. *Arq Bras Oftalmol*. 2013;76(3):170–4. <https://doi.org/10.1590/S0004-27492013000300008>.
68. Więckowska B, Kubiak KB, Jóźwiak P, Moryson W, Stawińska-Witoszyńska B. Cohen's kappa coefficient as a measure to assess classification improvement following the addition of a new marker to a regression model. *Int J Environ Res Public Health*. 2022. <https://doi.org/10.3390/ijerph191610213>.
69. Dj Novakovi J, Veljovi A, Ili SS, Papi Z, Tomovi M. Evaluation of classification models in machine learning. 2017.
70. Chicco D, Warrens MJ, Jurman G. The Matthews correlation coefficient (MCC) is more informative than Cohen's kappa and Brier score in binary classification assessment. *IEEE Access*. 2021;9:78368–81. <https://doi.org/10.1109/ACCESS.2021.3084050>.
71. Clark JA, Harman M, Clark J. Metrics are fitness functions too. 2004. <https://doi.org/10.1109/METRICS.2004.30>.
72. Singh DAAG, Balamurugan SAA, Leavline EJ. Performance metrics for feature selection, clustering and classification algorithms. *Asian J Res Soc Sci Humanit*. 2016;6(11):107. <https://doi.org/10.5958/2249-7315.2016.01179.5>.
73. Vujović ŽD. Classification model evaluation metrics. [Online]. Available: www.ijacsa.thesai.org.
74. Shahapurkar AS. Accurate fraud detection in credit card transactions using hybrid heuristic and meta-heuristic algorithms. [Online]. Available: <https://ssrn.com/abstract=3834947>.
75. Illeberi E, Sun Y, Wang Z. A machine learning based credit card fraud detection using the GA algorithm for feature selection. *J Big Data*. 2022. <https://doi.org/10.1186/s40537-022-00573-8>.

76. Rawashdeh E, Al-Ramahi N, Ahmad H, Zaghloul R. Efficient credit card fraud detection using evolutionary hybrid feature selection and random weight networks. *Int J Data Netw Sci*. 2024;8(1):463–72. <https://doi.org/10.5267/j.ijdns.2023.9.009>.
77. Alex SA, Jesu Vedha Nayahi J, Kaddoura S. Deep convolutional neural networks with genetic algorithm-based synthetic minority over-sampling technique for improved imbalanced data classification. *Appl Soft Comput*. 2024. <https://doi.org/10.1016/j.asoc.2024.111491>.
78. Emmanuel I, Sun Y, Wang Z. A machine learning-based credit risk prediction engine system using a stacked classifier and a filter-based feature selection method. *J Big Data*. 2024. <https://doi.org/10.1186/s40537-024-00882-0>.
79. Pertiwi DAA, Ahmad K, Salahudin SN, Annegrat AM, Muslim MA. Using genetic algorithm feature selection to optimize XGBoost performance in Australian credit. *J Soft Comput Explor*. 2024;5(1):92–8. <https://doi.org/10.52465/josce.v5i1.302>.
80. Singh A, Jain A, Biabale SE. Financial fraud detection approach based on Firefly optimization algorithm and support vector machine. *Appl Comput Intell Soft Comput*. 2022. <https://doi.org/10.1155/2022/1468015>.
81. Derrac J, García S, Molina D, Herrera F. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm Evol Comput*. 2011;1(1):3–18. <https://doi.org/10.1016/j.swevo.2011.02.002>.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.